ΤΕΧΝΗΤΗ ΝΟΗΜΟΣΥΝΗ

2η ΕΡΓΑΣΙΑ

Μέρος Α' (υλοποίηση αλγορίθμων)

Επιλέξαμε να υλοποιήσουμε τους αλγορίθμους:

- Αφελής ταξινομητής Bayes (Naive Bayes) πολυμεταβλητή μορφή Bernoulli
- Τυχαίο Δάσος (Random Forest)
- Λογιστική Παλινδρόμηση (Logistic Regression) με στοχαστική ανάβαση κλίσης

για την κατάταξη κειμένων της βάσης δεδομένων IMDB σε δύο κατηγορίες: θετικές και αρνητικές κριτικές ταινιών. Η διαδικασία περιλαμβάνει προεπεξεργασία των δεδομένων, εκπαίδευση του μοντέλου, αξιολόγηση και ανάλυση καμπυλών μάθησης. Η εργασία έχει υλοποιηθεί σε Python Jupyter Notebook.

Τα δεδομένα αντλήθηκαν από το IMDB dataset του keras. Για την προεπεξεργασία των δεδομένων, χρησιμοποιούμε την συνάρτηση **`tf.keras.datasets.imdb.load_data()`** για να αντλήσουμε τα δεδομένα των κριτικών των ταινιών και να τα χωρίσουμε σε δεδομένα εκπαίδευσης και δεδομένα ελέγχου, όπου αγνοούνται οι n πιο συχνές λέξεις και οι k πιο σπάνιες λέξεις.

Οι κριτικές των ταινιών, αποθηκεύονται στους πίνακες x_train και x_test και η κατηγορία, στην οποία ανήκει κάθε κριτική της κάθε ταινίας, στους πίνακες y_train και y_test, καθένας από αυτούς τους πίνακες έχει μέγεθος 25000. Οι πίνακες y_train και y_test περιέχουν 0 (που αναπαριστά την αρνητική κριτική) και 1 (που αναπαριστά την θετική κριτική).

Αρχικά οι πίνακες x_train και x_test περιέχουν τις κριτικές σε μορφή ακολουθίας ακεραίων, όπου ο κάθε ακέραιος αντιστοιχεί σε μία λέξη:

```
x_train[0] = [1, 14, 22, 16, 43, 530, 973, ...]
```

Στη συνέχεια, τροποποιούμε τους πίνακες x_train και x_test, έτσι ώστε αντί να περιέχουν μία λίστα με αριθμούς για κάθε κριτική, να περιέχουν συμβολοσειρές λέξεων.

```
x_train[0] = "the movie was excellent fantastic storyline"
```

Τέλος, για να μπορούμε να αναλύσουμε τα δεδομένα μας με έναν δυαδικό ταξινομητή, μετατρέπουμε το κείμενο σε δυαδικά διανύσματα με την χρήση του CountVectorizer, έτσι κάθε διάνυσμα αντιστοιχεί σε μια κριτική και κάθε τιμή αντιστοιχεί σε μία λέξη του λεξιλογίου μας που είναι 1 αν η συγκεκριμένη λέξη υπάρχει στην συγκεκριμένη κριτική ή 0 αν δεν υπάρχει.

x_train_binary[0] = [1, 0, 1, 0, 1, ...] # Δυαδικός πίνακας για κάθε λέξη στο λεξικό.

Υλοποίηση Bernoulli Naive Bayes

Οι τιμές των υπερπαραμέτρων που χρησιμοποιήσαμε είναι:

m = 3000: Μέγιστο μέγεθος λεξιλογίου.

n = 50: Αποκλεισμός των 50 πιο συχνών λέξεων.

k = 20: Αποκλεισμός των 20 λιγότερο συχνών λέξεων.

Αυτές οι τιμές προέκυψαν από μετρήσεις ως εξής, στην υλοποίησή μας υπάρχει αλγόριθμος που δοκιμάζει διαφορετικές τιμές των υπερπαραμέτρων m, n, k για να βρει τον συνδυασμό που δίνει τη μεγαλύτερη ακρίβεια (validation accuracy). Οι δυνατές τιμές των m, n, k που δοκιμάζονται είναι: m = [500, 2000, 2500, 3000], n = [50, 100, 200, 300], k = [0, 20, 50, 80]. Αυτόν τον αλγόριθμο τον εκτελέσαμε μία φορά έτσι ώστε να προκύψουν οι βέλτιστες τιμές των υπερπαραμέτρων, και στην συνέχεια τον απενεργοποιήσαμε (hyperparameter_tuning = False).

Επίσης αναπτύχθηκε αλγόριθμος για **Information Gain**, ο οποίος εξετάζει το κέρδος πληροφορίας. Μετά από δοκιμές προέκυψε ότι ο αλγόριθμος αυτός ενδεχομένως επιβάρυνε την εκτέλεση του κώδικα χωρίς να προσφέρει ουσιαστική βελτίωση, έτσι για τις τελικές μετρήσεις τον απενεργοποιήσαμε (infogain = False).

Μέθοδοι:

fit(self, X, y)

Είναι η **μέθοδος εκπαίδευσης**, δηλαδή μαθαίνει τις πιθανότητες από τα δεδομένα εκπαίδευσης. Αρχικά, υπολογίζει τις εκ των προτέρων (a-priori) πιθανότητες των κλάσεων και τις λογαριθμίζει. Έπειτα, υπολογίζει τις πιθανότητες κάθε λέξης (χαρακτηριστικού) ανά κατηγορία (οι κατηγορίες είναι δύο, 0: αρνητική αξιολόγηση, 1: θετική αξιολόγηση).

Οι πιθανότητες που ως εξής: Αρχικά αθροίζουμε κάθε στήλη του πίνακα με τα παραδείγματα εκπαίδευσης και καταλήγουμε με ένα διάνυσμα που περιέχει το πόσες φορές εμφανίστηκε κάθε χαρακτηριστικό σε μία συγκεκριμένη κατηγορία. Στη συνέχεια μετατρέπουμε κάθε τιμή του διανύσματος από άθροισμα σε πιθανότητα εμφάνισης του κάθε χαρακτηριστικού δεδομένης μίας κλάσης, διαιρώντας το αποτέλεσμα του βήματος 1 με τη συνολική ποσότητα της κλάσης εφαρμόζοντας Laplace smoothing για αποφυγή μηδενικών πιθανοτήτων.

Έτσι καταλήγουμε με δύο διανύσματα, ένα για κάθε κατηγορία, που το καθένα περιέχει την πιθανότητα εμφάνισης ενός χαρακτηριστικού δεδομένου ότι ένα παράδειγμα με αυτό το χαρακτηριστικό ανήκει στην κατηγορία αυτή.

Ομοίως, υπολογίζουμε και δύο διανύσματα για κάθε κατηγορία, p_log_pos_feature0 και p_log_neg_feature0, που το καθένα περιέχει την πιθανότητα μη εμφάνισης ενός χαρακτηριστικού (δηλαδή το διάνυσμα χαρακτηριστικών να έχει τιμή 0 για αυτό το χαρακτηριστικό), δεδομένου ότι ένα παράδειγμα χωρίς αυτό το χαρακτηριστικό ανήκει στην κατηγορία αυτή.

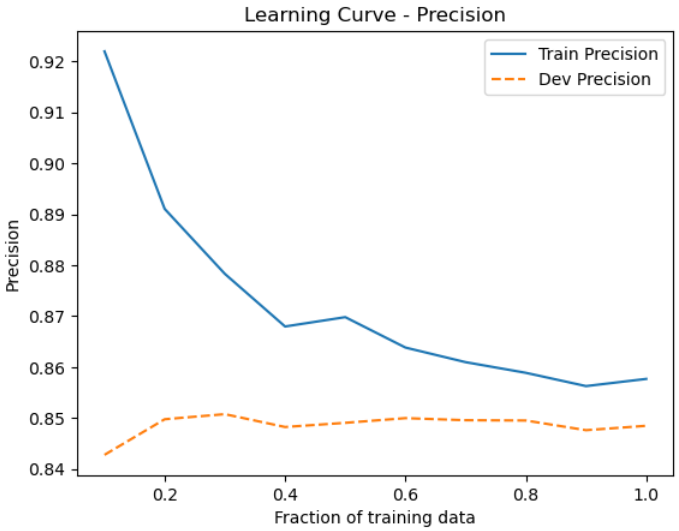
Αποθηκεύουμε τα αποτελέσματα αυτά και μπορούμε να τα χρησιμοποιήσουμε για την ταξινόμηση κριτικών.

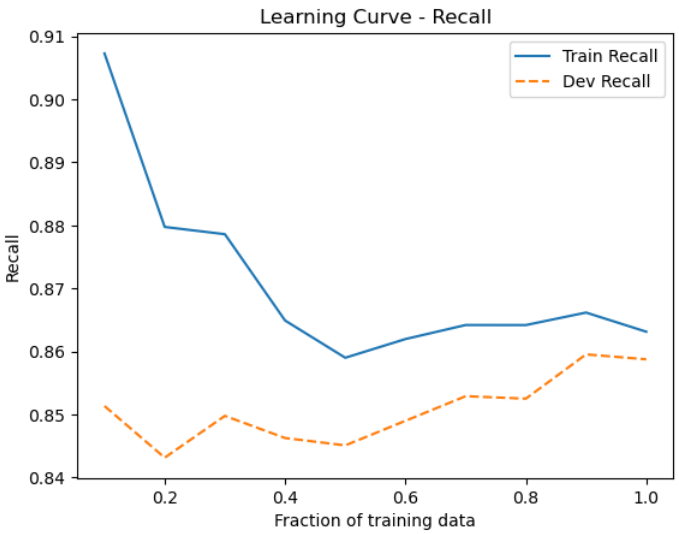
predict(self, X)

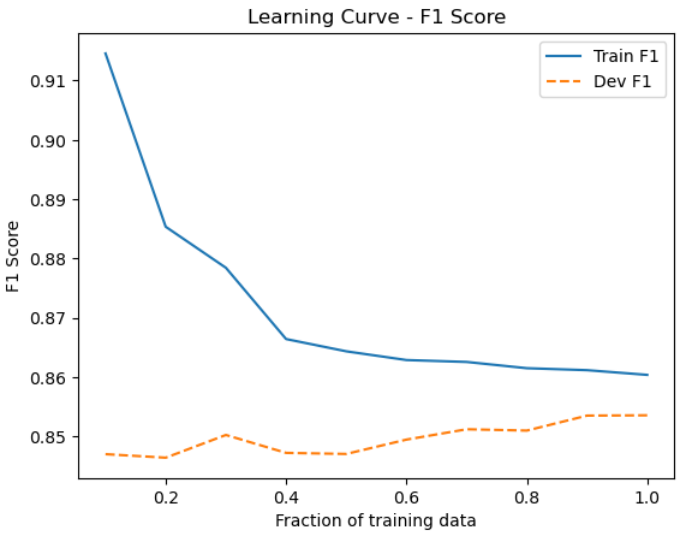
Είναι η μέθοδος **που κάνει προβλέψεις** για νέα δεδομένα. Μετατρέπει τις εισόδους σε δυαδικές τιμές (0 = απουσία λέξης, 1 = παρουσία λέξης), χρησιμοποιεί τον κανόνα του Bayes για να υπολογίσει τις πιθανότητες για κάθε κατηγορία (θετική/αρνητική κριτική), συγκρίνει τις πιθανότητες και επιστρέφει την κατηγορία με τη μεγαλύτερη πιθανότητα.

Καμπύλες μάθησης

Οι παρακάτω καμπύλες μάθησης δείχνουν αποτελέσματα ακρίβειας (precision), ανάκλησης (recall) και F1, για την κατηγορία των θετικών αξιολογήσεων, στα δεδομένα εκπαίδευσης (training data) και ανάπτυξης (development data) συναρτήσει του πλήθους των παραδειγμάτων εκπαίδευσης που χρησιμοποιούνται:







Παρατηρούμε μία παρόμοια συμπεριφορά και στα τρία παραπάνω διαγράμματα:

- Οι καμπύλες Train ξεκινούν από αρκετά υψηλές τιμές και μειώνονται σταδιακά όσο αυξάνεται το μέγεθος των δεδομένων εκπαίδευσης.
- Οι καμπύλες Dev παραμένουν σταθερές ή ελαφρώς βελτιώνονται (ιδιαίτερα το Recall).
- Καθώς το μέγεθος του training set αυξάνεται, η διαφορά μεταξύ Train και Dev καμπυλών μειώνεται.

Έτσι μπορούμε να συμπεράνουμε ότι η **γενίκευση του μοντέλου βελτιώνεται**, αλλά η συνολική του απόδοση φαίνεται να σταθεροποιείται σε ένα συγκεκριμένο επίπεδο.

Πίνακες:

```
Test set evaluation:
              precision    recall  f1-score   support

           0       0.85      0.85      0.85     12500
           1       0.85      0.85      0.85     12500

    accuracy                           0.85     25000
   macro avg       0.85      0.85      0.85     25000
weighted avg       0.85      0.85      0.85     25000

Precision (class-wise): [0.84606177 0.8477825 ]
Recall (class-wise): [0.84816 0.84568]
F1-score (class-wise): [0.84710958 0.84672995]
Micro-averaged metrics (Precision, Recall, F1): (0.84692, 0.84692, 0.84692)
Macro-averaged metrics (Precision, Recall, F1): (0.8469221337098911, 0.84692, 0.84691976462383)
```

Από τα αποτελέσματα της αξιολόγησης στο Test Set, μπορούμε να βγάλουμε τα εξής συμπεράσματα:

- Σταθερή απόδοση σε όλες τις μετρικές. Accuracy 85%, δηλαδή το μοντέλο έχει 85% ποσοστό επιτυχίας στην ταξινόμηση των δειγμάτων. Precision, Recall, και F1-score είναι όλα ίσα με 0.85. Αυτό δείχνει ότι το μοντέλο είναι ισορροπημένο και δεν υπάρχει σοβαρό trade-off μεταξύ precision και recall.
- Ισορροπία μεταξύ των κατηγοριών. Οι επιδόσεις είναι σχεδόν ίδιες και για τις δύο κατηγορίες (0 και 1), με μικρές διαφορές: Precision: 0.846 για την κλάση 0 και 0.848 για την κλάση 1, Recall: 0.848 για την κλάση 0 και 0.846 για την κλάση 1, F1-score: 0.847 και 0.846 αντίστοιχα. Έτσι μπορούμε να πούμε ότι το μοντέλο δεν ευνοεί κάποια συγκεκριμένη κατηγορία, κάτι που δείχνει ισορροπημένη ταξινόμηση.
- Macro-averaged Precision, Recall, και F1-score είναι όλα 0.8469, που σημαίνει ότι οι κλάσεις έχουν περίπου ίση σημασία.
- Micro-averaged μετρικές είναι επίσης 0.8469, κάτι που δείχνει ότι το μοντέλο έχει σταθερή απόδοση ανεξαρτήτως των συχνοτήτων των κατηγοριών.

Υλοποίηση Random Forest

Οι τιμές των υπερπαραμέτρων που χρησιμοποιήσαμε είναι:

m = 500: Μέγιστο μέγεθος λεξιλογίου.

n = 50: Αποκλεισμός των 50 πιο συχνών λέξεων.

k = 20: Αποκλεισμός των 20 λιγότερο συχνών λέξεων.

Αυτές οι τιμές προέκυψαν από μετρήσεις ως εξής, στην υλοποίησή μας υπάρχει αλγόριθμος που δοκιμάζει διαφορετικές τιμές των υπερπαραμέτρων m, n, k για να βρει τον συνδυασμό που δίνει τη μεγαλύτερη ακρίβεια (validation accuracy). Οι δυνατές τιμές των m, n, k που δοκιμάζονται είναι: m = [500, 2000, 2500, 3000], n = [50, 100, 200, 300], k = [0, 20, 50, 80]. Αυτόν τον αλγόριθμο τον εκτελέσαμε μία φορά έτσι ώστε να προκύψουν οι βέλτιστες τιμές των υπερπαραμέτρων, και στην συνέχεια τον απενεργοποιήσαμε (hyperparameter_tuning = False).

Επίσης αναπτύχθηκε αλγόριθμος για **Information Gain**, ο οποίος εξετάζει το κέρδος πληροφορίας. Μετά από δοκιμές προέκυψε ότι ο αλγόριθμος αυτός ενδεχομένως επιβάρυνε την εκτέλεση του κώδικα χωρίς να προσφέρει ουσιαστική βελτίωση, έτσι για τις τελικές μετρήσεις τον απενεργοποιήσαμε (infogain = False).

Μέθοδοι:

fit(self, X, y)

Είναι η μέθοδος η οποία εκπαιδεύει τον αλγόριθμο Random Forest χρησιμοποιώντας τα δεδομένα εκπαίδευσης X και τις ετικέτες y. Αρχικά εκκαθαρίζει την προηγούμενη λίστα δέντρων. Στην συνέχεια για κάθε δέντρο:

- Επιλέγει τυχαία δείγματα (random_x, random_y).
- Επιλέγει τυχαία χαρακτηριστικά (random_features).
- Δημιουργεί και εκπαιδεύει ένα δέντρο ID3 χρησιμοποιώντας τα τυχαία δείγματα (χρησιμοποιήθηκε το ID3 των φροντιστηρίων).
- Προσθέτει το εκπαιδευμένο δέντρο στη λίστα random_trees.

select_random_samples(self, X, y)

Επιλέγει τυχαία δείγματα δεδομένων από το X και τις ετικέτες y μέσω δειγματοληψίας με επανάθεση (bootstrap sampling). Επιστρέφει τα τυχαία δείγματα (random_x) και τις αντίστοιχες ετικέτες (random_y).

select_random_features(self, random_x)

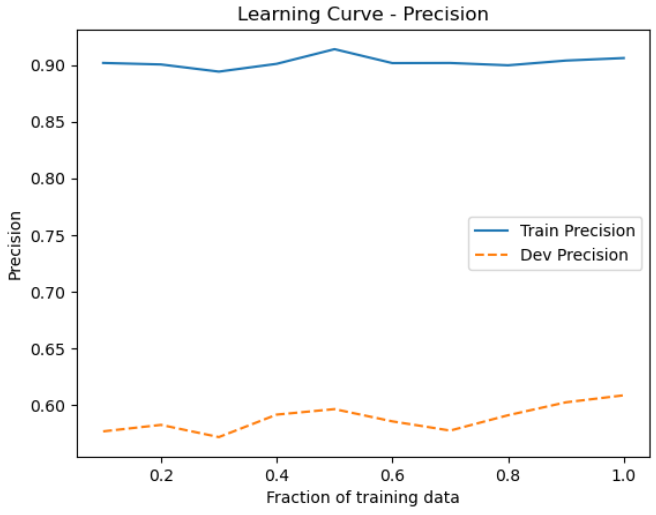
Επιλέγει τυχαία έναν αριθμό m χαρακτηριστικών από το σύνολο των διαθέσιμων χαρακτηριστικών και τα επιστρέφει ως πίνακα.

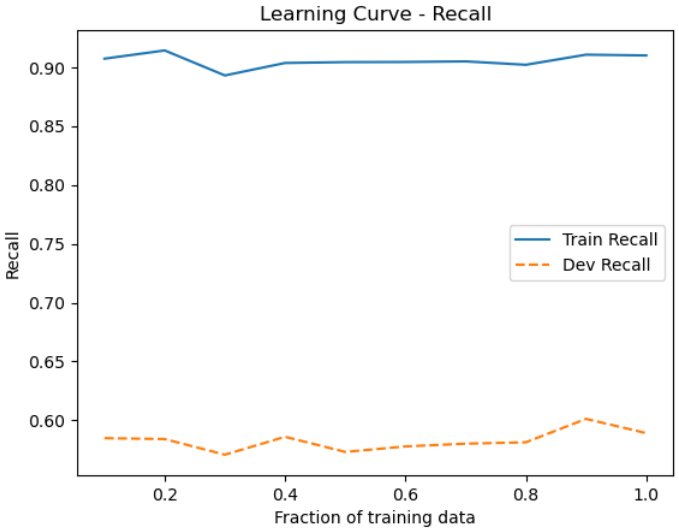
predict(self, X)

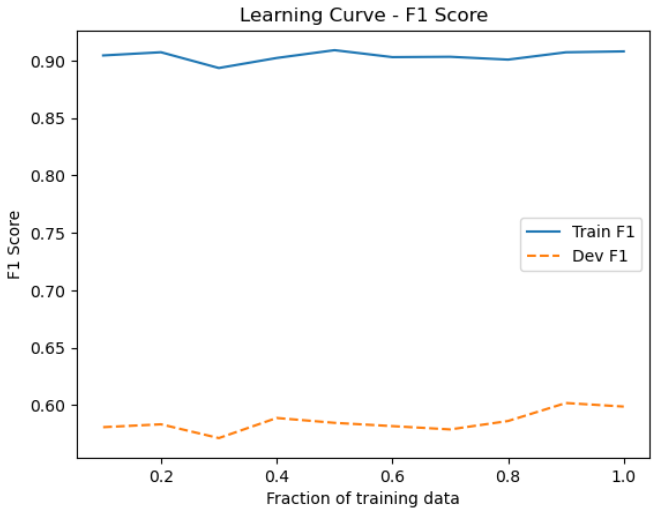
Είναι η μέθοδος **που κάνει προβλέψεις** για νέα δεδομένα. Αποφασίζει και κατατάσσει κάθε παράδειγμα εκπαίδευσης στην κατηγορία την οποία πιστεύει ότι ανήκει κάθε κριτική.

Καμπύλες μάθησης

Οι παρακάτω καμπύλες μάθησης δείχνουν αποτελέσματα ακρίβειας (precision), ανάκλησης (recall) και F1, για την κατηγορία των θετικών αξιολογήσεων, στα δεδομένα εκπαίδευσης (training data) και ανάπτυξης (development data) συναρτήσει του πλήθους των παραδειγμάτων εκπαίδευσης που χρησιμοποιούνται:







Παρατηρούμε μία παρόμοια συμπεριφορά και στα τρία παραπάνω διαγράμματα:

- Η καμπύλη εκπαίδευσης παραμένει υψηλή (~0.9), πράγμα που σημαίνει ότι το μοντέλο έχει υψηλή ακρίβεια στα δεδομένα εκπαίδευσης (δηλαδή βρίσκει τις θετικές περιπτώσεις με λίγα λάθη). Η καμπύλη εκπαίδευσης διατηρεί υψηλή απόδοση (~0.9), υποδεικνύοντας ότι το μοντέλο βρίσκει αρκετές σωστές θετικές περιπτώσεις στα δεδομένα εκπαίδευσης. Ομοίως κα η καμπύλη εκπαίδευσης (Train F1) παραμένει αρκετά υψηλή (~0.9) σε όλο το εύρος των δεδομένων εκπαίδευσης.
- Ωστόσο η καμπύλη αξιολόγησης είναι αρκετά χαμηλότερη (~0.55-0.65), με σταθερή απόδοση. Αυτό δείχνει ότι το μοντέλο δεν μπορεί να εξασφαλίσει ακρίβεια στα δεδομένα επικύρωσης. Η καμπύλη αξιολόγησης παραμένει χαμηλή και σχετικά σταθερή, με μικρές διακυμάνσεις, γεγονός που δείχνει ότι το μοντέλο δεν γενικεύει καλά για την ανίχνευση των θετικών περιπτώσεων στα δεδομένα επικύρωσης. Η καμπύλη αξιολόγησης (Dev F1) είναι αισθητά χαμηλότερη, κυμαίνεται γύρω στο 0.55-0.65, γεγονός που υποδεικνύει πιθανή υπερπροσαρμογή (overfitting). Το μοντέλο μαθαίνει πολύ καλά τα δεδομένα εκπαίδευσης, αλλά δεν αποδίδει εξίσου καλά στα δεδομένα επικύρωσης.

Συμπερασματικά μπορούμε να πούμε ότι ο Random Forest παρουσιάζει ισχυρή ένδειξη overfitting. Το μοντέλο αποδίδει εξαιρετικά καλά στα δεδομένα εκπαίδευσης αλλά δεν γενικεύει στα δεδομένα επικύρωσης. Πιθανές λύσεις περιλαμβάνουν τη μείωση του βάθους των δέντρων, την αύξηση των δεδομένων εκπαίδευσης ή την προσθήκη κανονικοποίησης (regularization).

Πίνακες:

```
Test set evaluation:
              precision    recall  f1-score   support

           0       0.58      0.57      0.58     12500
           1       0.58      0.60      0.59     12500

    accuracy                           0.58     25000
   macro avg       0.58      0.58      0.58     25000
weighted avg       0.58      0.58      0.58     25000

Precision (class-wise): [0.58487769 0.58013842]
Recall (class-wise): [0.56808 0.5968 ]
F1-score (class-wise): [0.57635648 0.58835128]
Micro-averaged metrics (Precision, Recall, F1): (0.58244, 0.58244, 0.58244)
Macro-averaged metrics (Precision, Recall, F1): (0.5825080558127437, 0.5824400000000001, 0.5823538773601089)
```

Από τα αποτελέσματα της αξιολόγησης στο Test Set, μπορούμε να βγάλουμε το συμπέρασμα ότι το μοντέλο παρουσιάζει χαμηλή έως μέτρια απόδοση (58%), κάτι που ίσως οφείλεται σε υπερπροσαρμογή (overfitting)

Υλοποίηση Logistic Regression

Η υλοποίηση του αλγορίθμου λογιστικής παλινδρόμησης για δυαδική ταξινόμηση, χρησιμοποιώντας τη στοχαστική ανάβαση κλίσης (SGD) και κανονικοποίηση (regularization)

Μέθοδοι:

__init__()

Αρχικοποιεί τις παραμέτρους της κλάσης: lamda_value: Ο συντελεστής κανονικοποίησης (regularization), max_epochs: Μέγιστος αριθμός εποχών εκπαίδευσης, eta: Ρυθμός μάθησης (learning rate), tolerance: Όριο ανοχής στη μεταβολή της απώλειας (loss), max_no_improvement: Μέγιστος αριθμός εποχών χωρίς βελτίωση στην απώλεια πριν από την τερματισμό της εκπαίδευσης.

predict()

Υπολογίζει τις προβλέψεις του μοντέλου για τα δεδομένα εισόδου X: Προσθέτει έναν όρο bias σε κάθε δείγμα, για κάθε δείγμα, υπολογίζει το εσωτερικό γινόμενο μεταξύ των χαρακτηριστικών και των βαρών και επιστρέφει 1 αν η πρόβλεψη είναι θετική και 0 αν είναι αρνητική

prob_pos()

Υπολογίζει την πιθανότητα ένα δείγμα να ανήκει στην κατηγορία 1 χρησιμοποιώντας τη σιγμοειδή συνάρτηση (sigmoid):

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

fit()

Εκπαιδεύει το μοντέλο χρησιμοποιώντας στοχαστική ανάβαση κλίσης (SGD) ως εξής:

Προετοιμάζει τα δεδομένα προσθέτοντας όρο bias και αρχικοποίηση των βαρών. Στην συνέχεια, επαναλαμβάνει ανά εποχή το να υπολογίζει την πιθανότητα prob_pos και να ενημερώνει τα βάρη με βάση την απώλεια, υπολογίζει την απώλεια για την τρέχουσα εποχή

Τέλος εκτελείται **έλεγχος σύγκλισης** με τον οποίο τερματίζει την εκπαίδευση αν η απώλεια δεν βελτιώνεται για έναν προκαθορισμένο αριθμό εποχών.

Επίσης εκτελείται τακτική **κανονικοποίηση** (Regularization) που ελέγχει την πολυπλοκότητα του μοντέλου και βοηθά στην αποφυγή υπερπροσαρμογής. Το βάρος κάθε χαρακτηριστικού μειώνεται με την προσθήκη του όρου:

$$w \leftarrow (1 - 2\lambda\eta)w + \eta(y - p(x))x$$

Early Stopping: Αν η απώλεια δεν βελτιωθεί για max_no_improvement εποχές, η εκπαίδευση τερματίζεται, βελτιστοποιώντας τον χρόνο εκπαίδευσης.

Πειραματική εύρεση της καλύτερης τιμής λ:

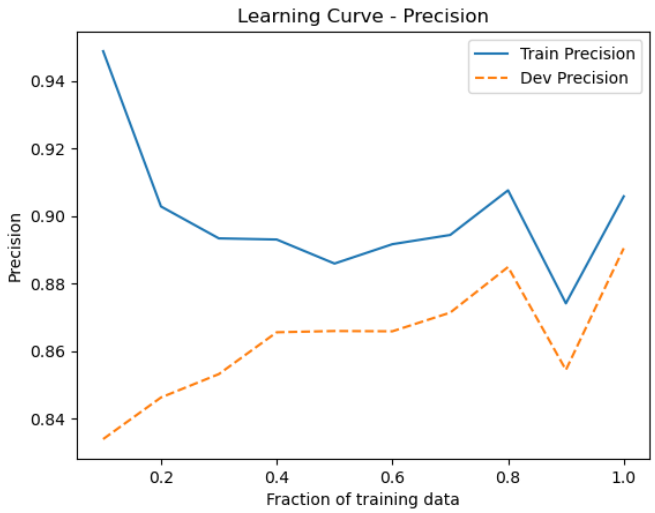
Για την υλοποίηση της λογιστικής παλινδρόμησης δημιουργήσαμε εκτός από το σύνολο εκπαίδευσης και ελέγχου, και ένα σύνολο επαλήθευσης για την εύρεση της τιμής της **σταθεράς κανονικοποίησης λ**, ως εξής:

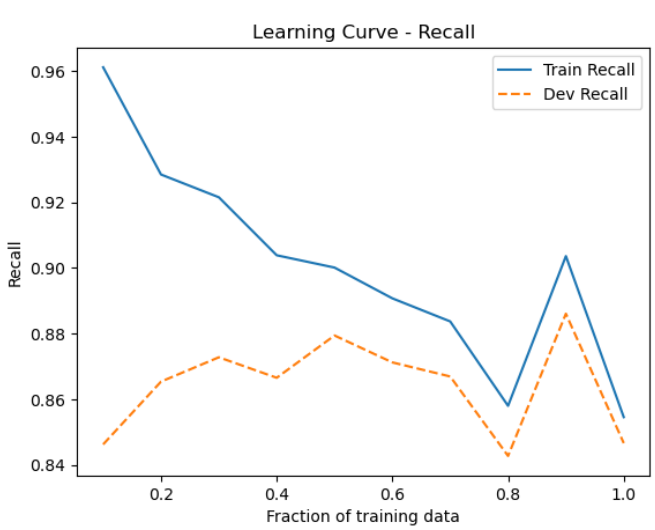
Μετά το φόρτωμα των δεδομένων από τη μέθοδο load_data, αποκόψαμε ένα μικρό κομμάτι από τα δεδομένα εκπαίδευσης (20% = 5000 αξιολογήσεις) με τη μέθοδο train_test_split του SKLearn. Στη συνέχεια, τα περάσαμε κι αυτά από τον CountVectorizer και τα χρησιμοποιήσαμε ως δεδομένα επαλήθευσης.

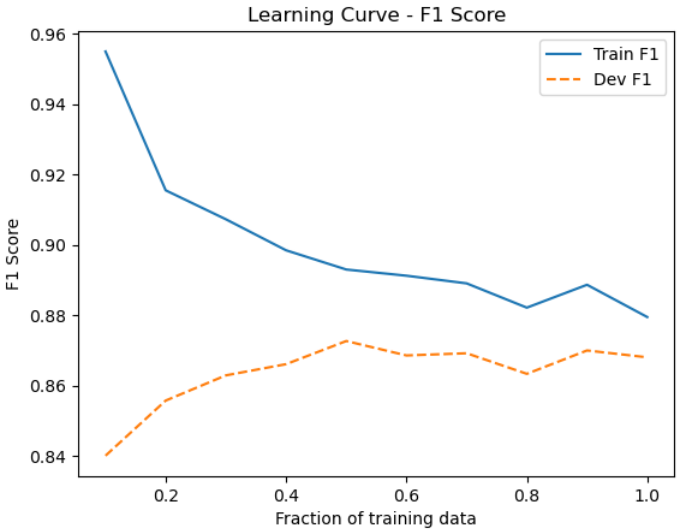
Έτσι προέκυψε ως καλύτερη τιμή του λ το 0.0005 (**Best λ: 0.0005 with Validation Accuracy: 0.868**). Στην συνέχεια αυτή η τιμή του λ χρησιμοποιήθηκε στο Training and Evaluating the Model.

Καμπύλες μάθησης

Οι παρακάτω καμπύλες μάθησης δείχνουν αποτελέσματα ακρίβειας (precision), ανάκλησης (recall) και F1, για την κατηγορία των θετικών αξιολογήσεων, στα δεδομένα εκπαίδευσης (training data) και ανάπτυξης (development data) συναρτήσει του πλήθους των παραδειγμάτων εκπαίδευσης που χρησιμοποιούνται:







Παρατηρώντας τα παραπάνω διαγράμματα βλέπουμε τα εξής:

Ας αναλύσουμε τις καμπύλες μάθησης για τον αλγόριθμο Logistic Regression που απεικονίζονται στις τρεις διαγράμματα:

Precision: η Train Precision παρουσιάζει πτώση, αλλά σταθεροποιείται μερικώς προς το τέλος. Η Dev Precision αρχικά αυξάνεται και παρουσιάζει ελαφρά αστάθεια με μεγάλες διακυμάνσεις στο τελευταίο μέρος των δεδομένων. Το Precision φαίνεται να επηρεάζεται από θόρυβο ή μη ισορροπημένα δεδομένα, γεγονός που θα μπορούσε να εξηγήσει τις διακυμάνσεις.

Recall: η Train Recall παρουσιάζει πτωτική τάση από υψηλές τιμές πάνω από 0.96. Η Dev Recall παρουσιάζει αστάθεια σε υψηλότερα ποσοστά εκπαίδευσης με μεγάλες αυξομειώσεις, κάτι που μπορεί να οφείλεται σε ευαισθησία του αλγορίθμου στη συγκεκριμένη μέτρηση.

F1 Score: η Train F1 ξεκινά υψηλά (γύρω στο 0.96) και μειώνεται σταδιακά με την αύξηση του δείγματος εκπαίδευσης. Η Dev F1 ξεκινά χαμηλά (γύρω στο 0.84) αλλά βελτιώνεται καθώς αυξάνεται το μέγεθος δεδομένων εκπαίδευσης. Οι τιμές τείνουν να συγκλίνουν χωρίς απόλυτη ταύτιση, υποδεικνύοντας καλή γενίκευση. Υπάρχει μικρό bias και μέτριο variance, αφού οι καμπύλες πλησιάζουν αλλά δεν συγκλίνουν απόλυτα.

Πίνακες:

```
Number of training examples = 25000 :: Converged at epoch 52 :: Loss = 0.30001025183738245
Test set evaluation:
              precision    recall  f1-score   support

           0       0.86      0.88      0.87     12500
           1       0.88      0.86      0.87     12500

    accuracy                           0.87     25000
   macro avg       0.87      0.87      0.87     25000
weighted avg       0.87      0.87      0.87     25000

Precision (class-wise): [0.86307054 0.87928355]
Recall (class-wise): [0.88192 0.86008]
F1-score (class-wise): [0.87239346 0.86957577]
Micro-averaged metrics (Precision, Recall, F1): (0.871, 0.871, 0.871)
Macro-averaged metrics (Precision, Recall, F1): (0.8711770461060431, 0.871, 0.8709846153798395)
```

Από τα αποτελέσματα της αξιολόγησης στο Test Set, μπορούμε να βγάλουμε το συμπέρασμα ότι ο αλγόριθμος Logistic Regression έχει καλή απόδοση με ισορροπημένη ακρίβεια και ευαισθησία και στις δύο κατηγορίες.

Μέρος Β' (σύγκριση των αλγορίθμων μας με τις υλοποιήσεις του Scikit-learn

Bernoulli Naive Bayes

Παρακάτω βλέπουμε την επίδοση της υλοποίησης του αλγορίθμου Bernoulli Naive Bayes, όπως τον υλοποιήσαμε εμείς, σε σχέση με τις επιδόσεις των Bernoulli Naive Bayes, Random Forest και Logistic Regression του Scikit-learn:

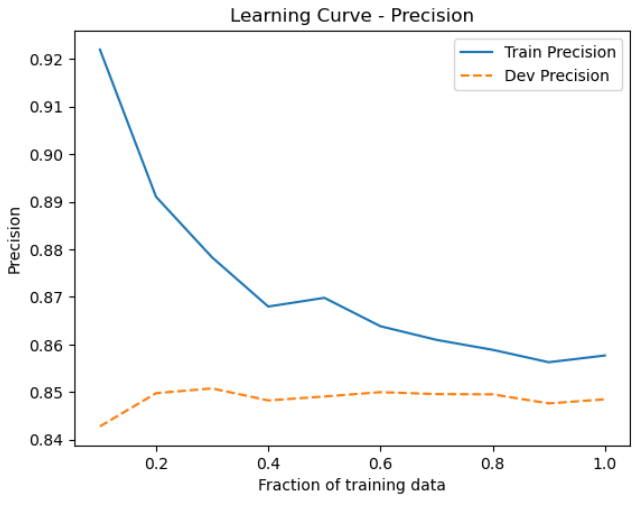
Bernoulli Naive Bayes against Scikit-learn Bernoulli Naive Bayes

```
Scikit-learn Bernoulli Naive Bayes - Test set evaluation:
              precision    recall  f1-score   support

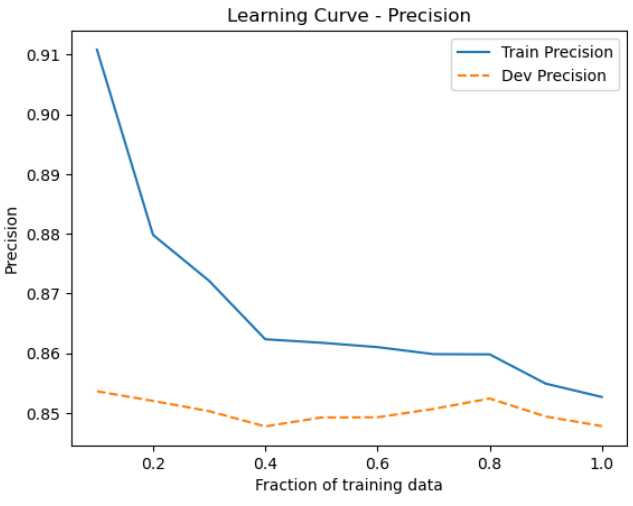
           0       0.85      0.85      0.85     12500
           1       0.85      0.85      0.85     12500

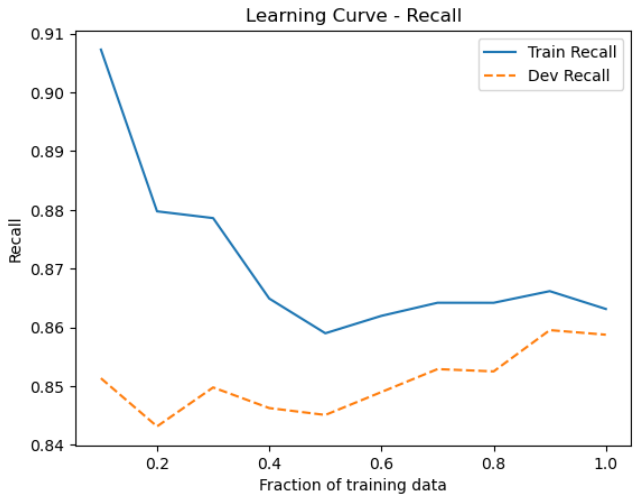
    accuracy                           0.85     25000
   macro avg       0.85      0.85      0.85     25000
weighted avg       0.85      0.85      0.85     25000
```

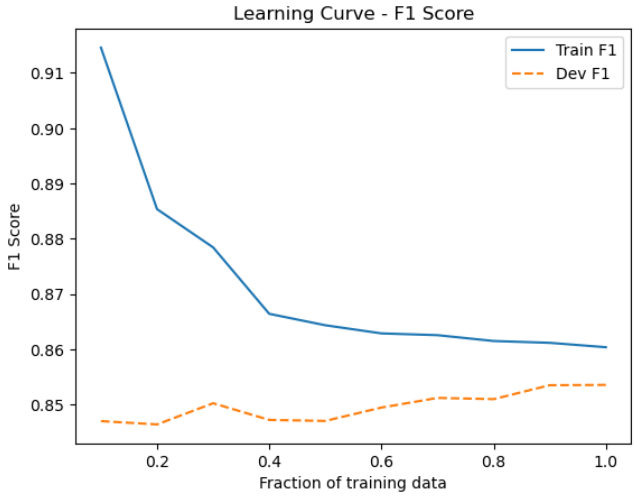
Bernoulli Naive Bayes

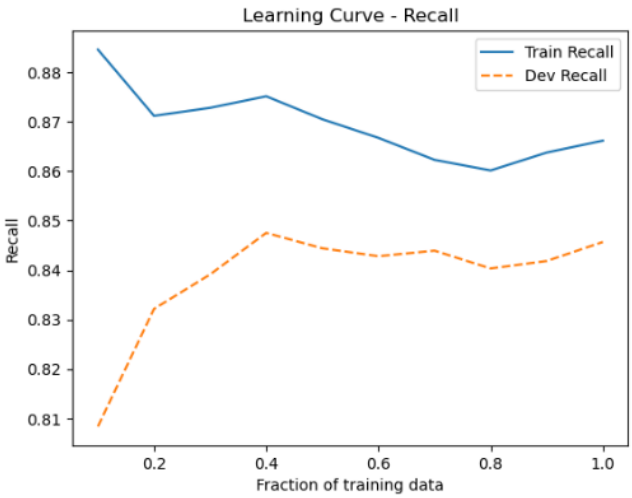


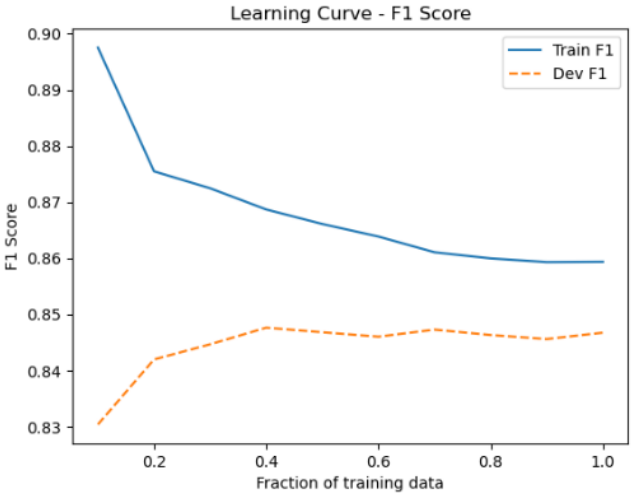
Scikit-learn Bernoulli Naive Bayes











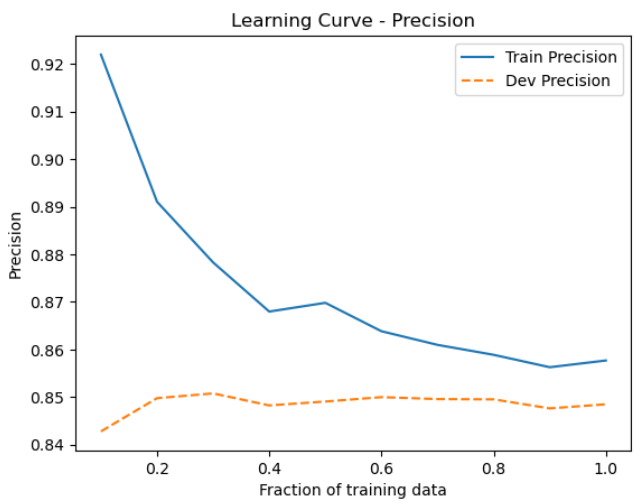
Bernoulli Naive Bayes against Scikit-learn Random Forest Classifier

```
Random Forest Classifier - Test set evaluation:
              precision    recall  f1-score   support

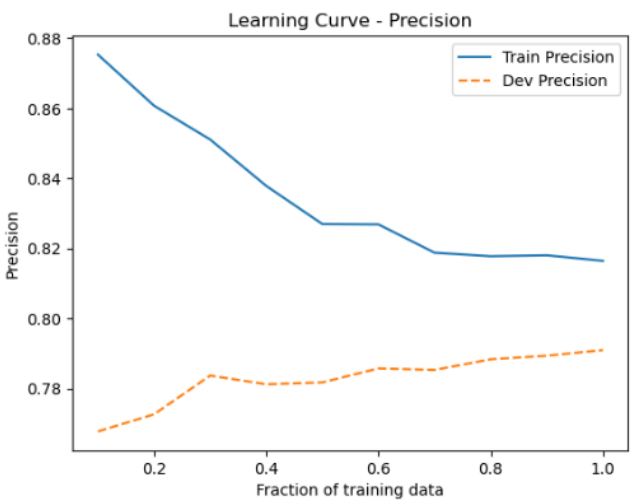
           0       0.85      0.77      0.81     12500
           1       0.79      0.87      0.83     12500

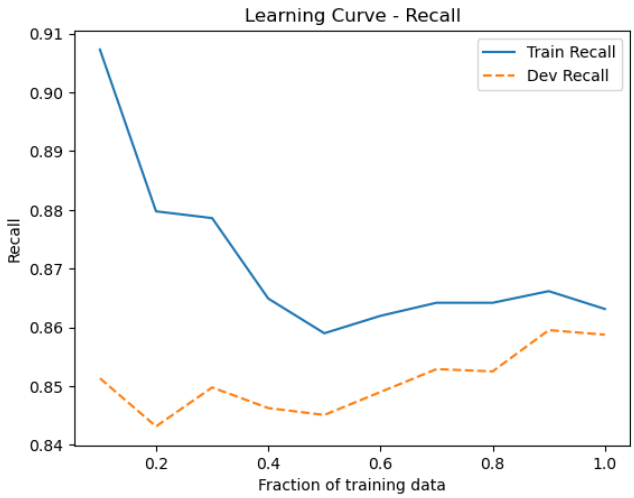
    accuracy                           0.82     25000
   macro avg       0.82      0.82      0.82     25000
weighted avg       0.82      0.82      0.82     25000
```

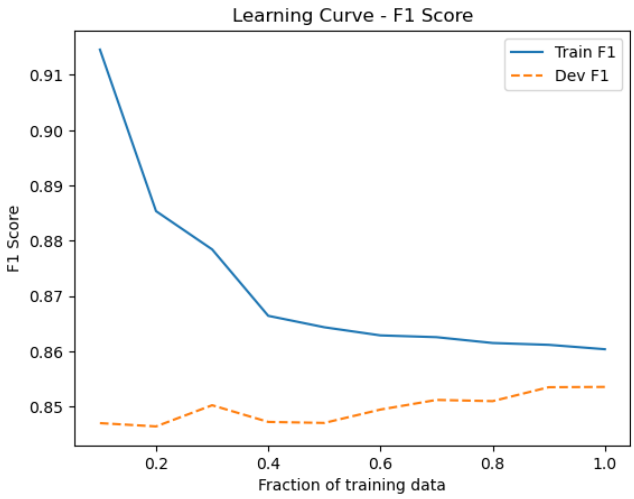
Bernoulli Naive Bayes

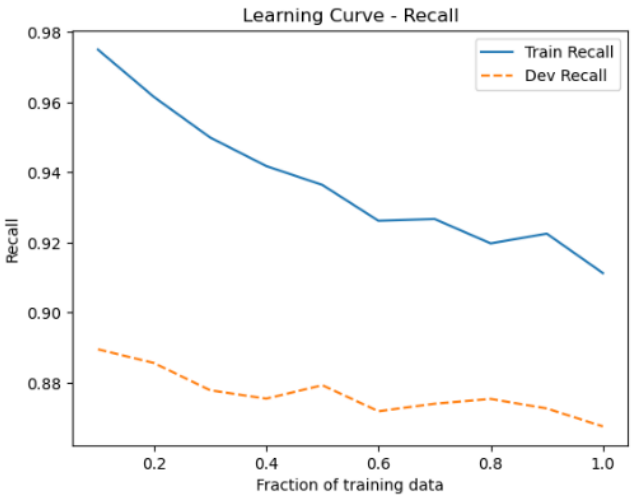


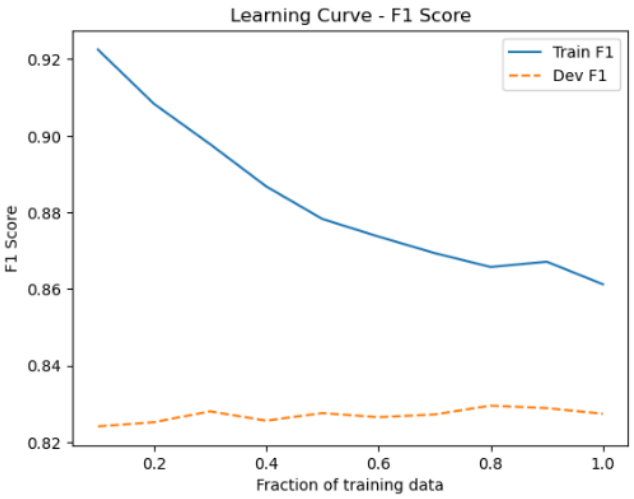
Scikit-learn Random Forest Classifier











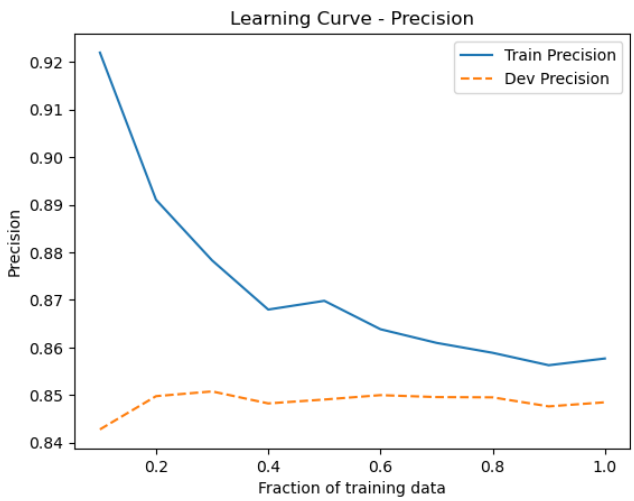
Bernoulli Naive Bayes against Scikit-learn Logistic Regression

```
Logistic Regression - Test set evaluation:
              precision    recall  f1-score   support

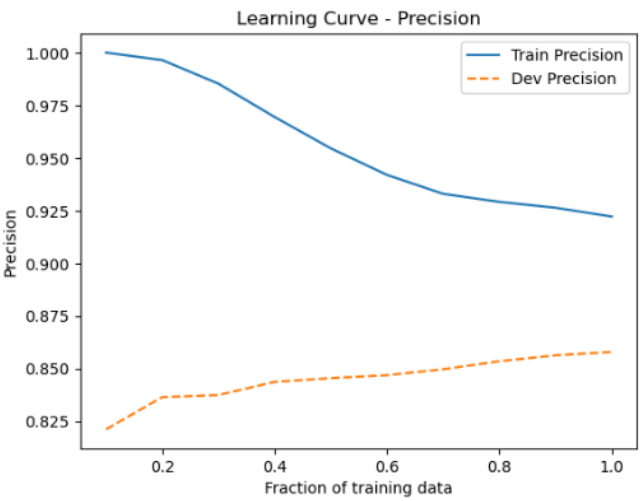
           0       0.86      0.86      0.86     12500
           1       0.86      0.86      0.86     12500

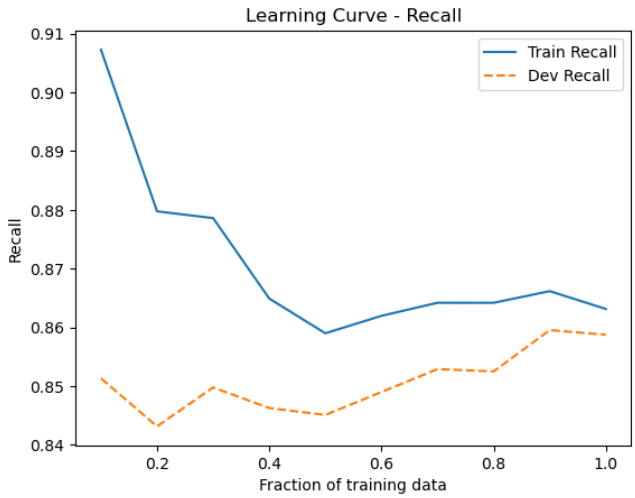
    accuracy                           0.86     25000
   macro avg       0.86      0.86      0.86     25000
weighted avg       0.86      0.86      0.86     25000
```

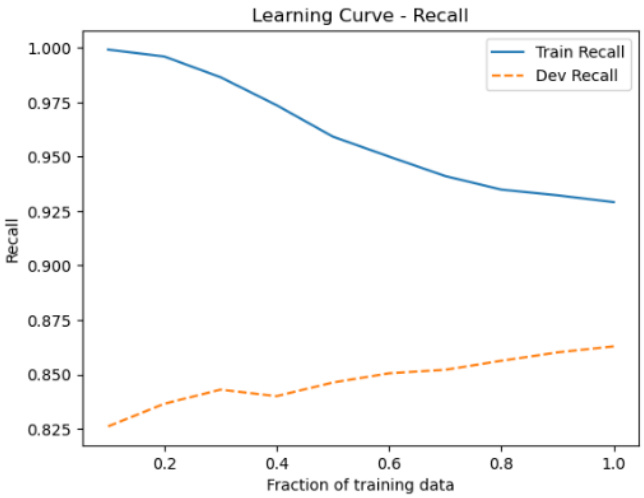
Bernoulli Naive Bayes

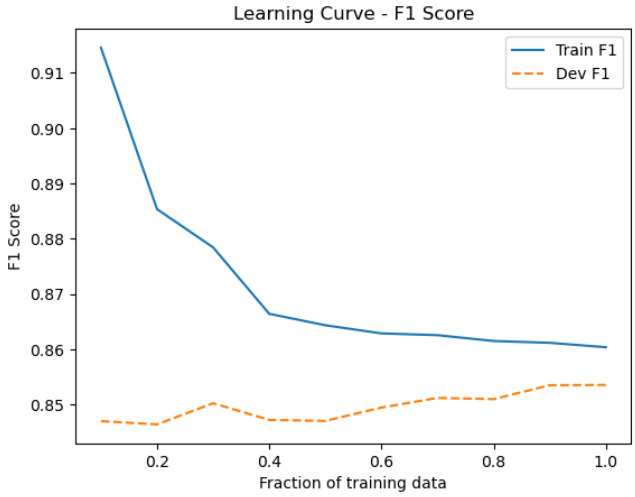


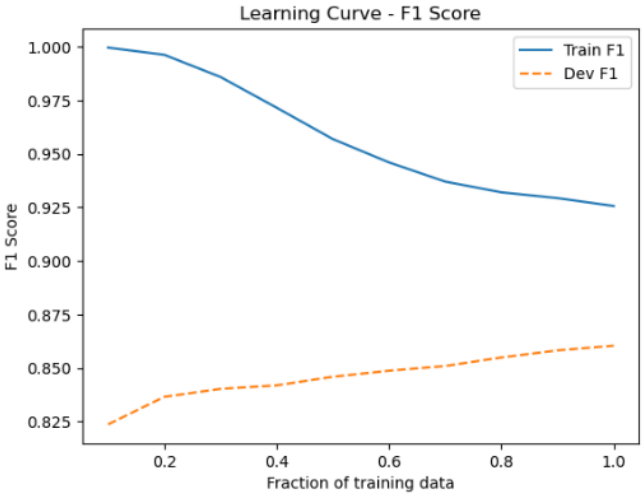
Scikit-learn Logistic Regression











Random Forest

Παρακάτω βλέπουμε την επίδοση της υλοποίησης του αλγορίθμου Random Forest, όπως τον υλοποιήσαμε εμείς, σε σχέση με τις επιδόσεις των Bernoulli Naive Bayes, Random Forest και Logistic Regression του Scikit-learn:

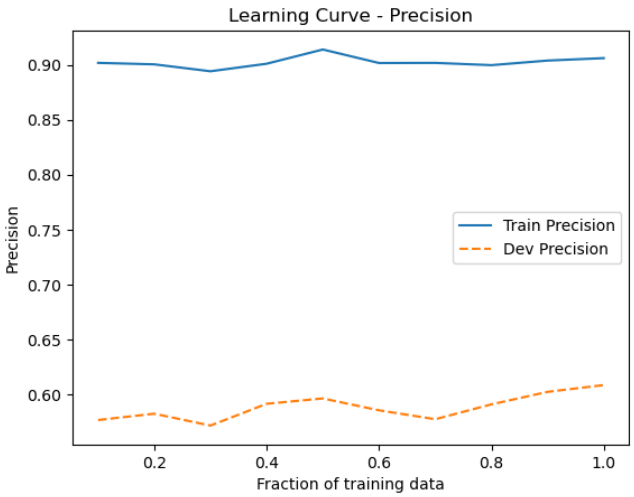
Random Forest against Scikit-learn Bernoulli Naive Bayes

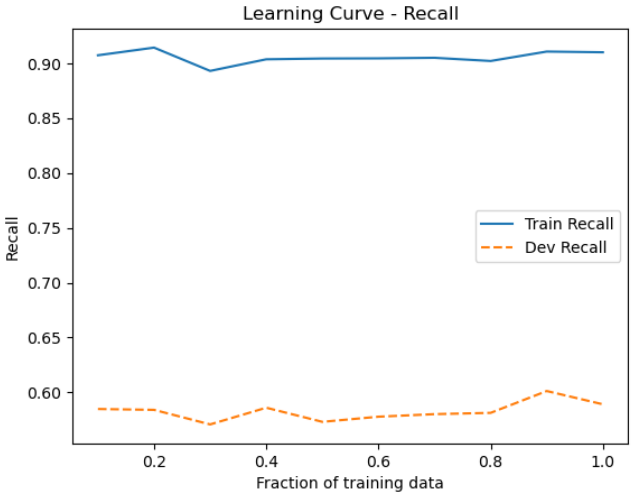
```
Scikit-learn Bernoulli Naive Bayes - Test set evaluation:
              precision    recall  f1-score   support

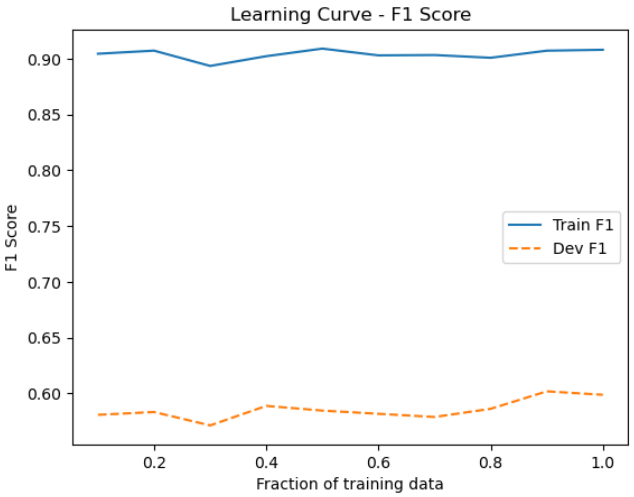
           0       0.85      0.85      0.85     12500
           1       0.85      0.85      0.85     12500

    accuracy                           0.85     25000
   macro avg       0.85      0.85      0.85     25000
weighted avg       0.85      0.85      0.85     25000
```

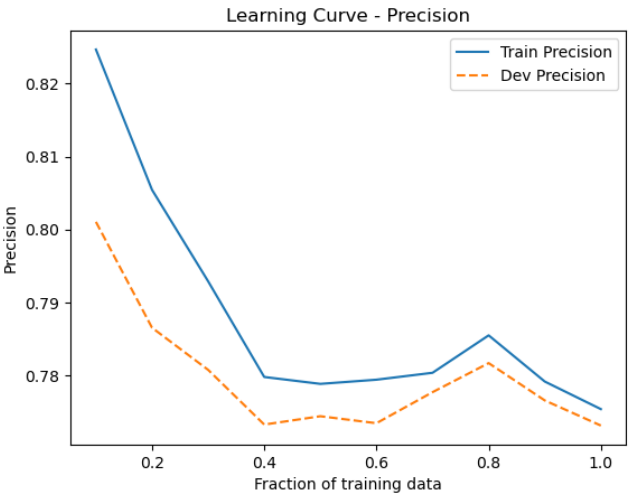
Random Forest

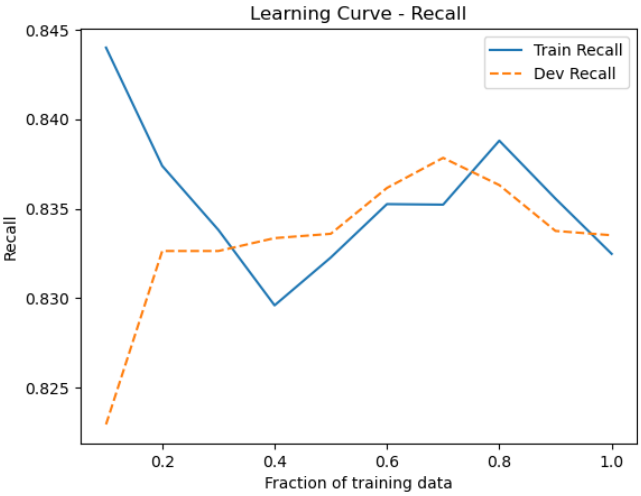


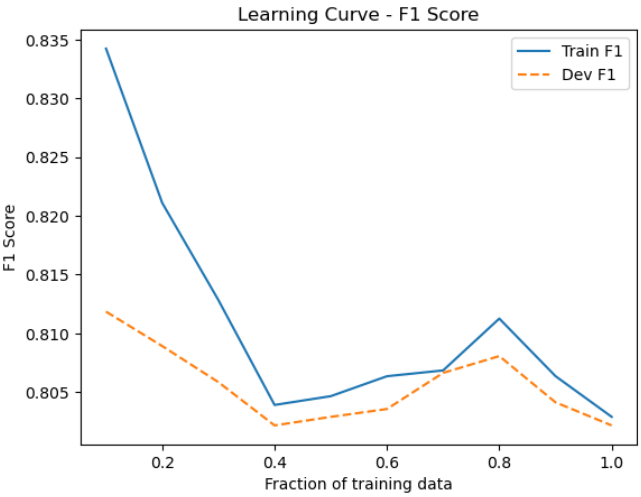




Scikit-learn Bernoulli Naive Bayes





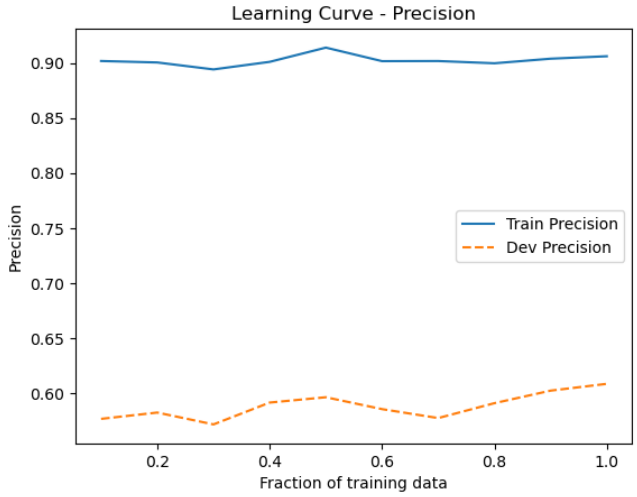


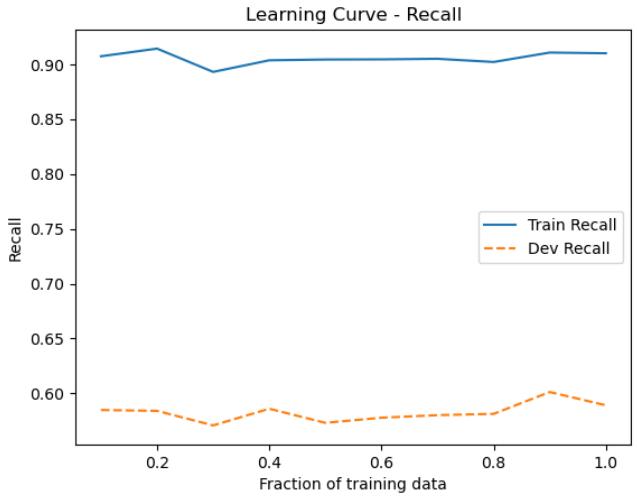
Random Forest against Scikit-learn Random Forest

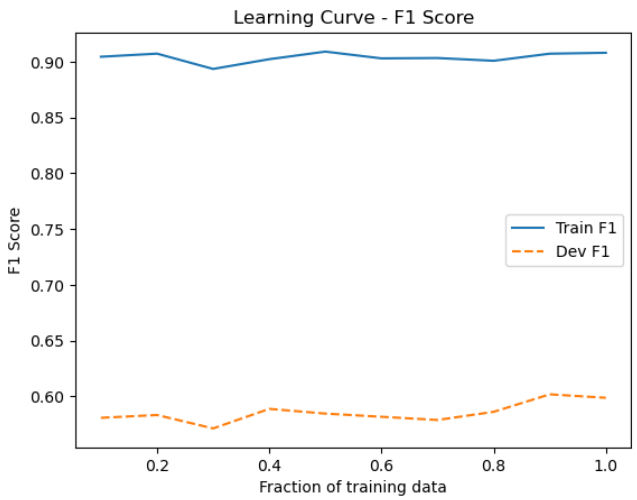
Random Forest Classifier - Test set evaluation:

	precision	recall	f1-score	support
0	0.83	0.68	0.75	12500
1	0.73	0.86	0.79	12500
accuracy			0.77	25000
macro avg	0.78	0.77	0.77	25000
weighted avg	0.78	0.77	0.77	25000

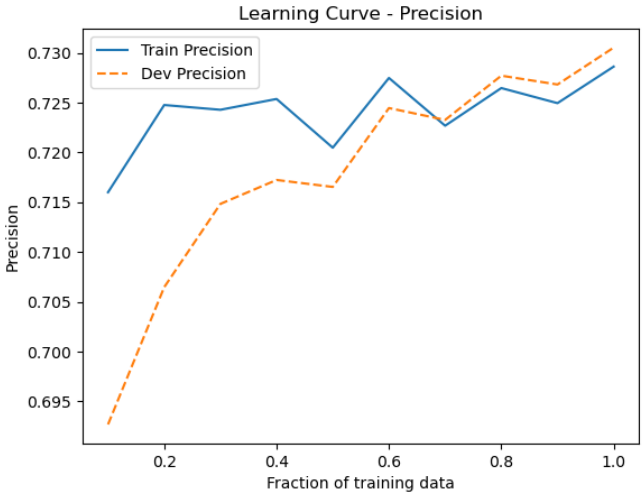
Random Forest

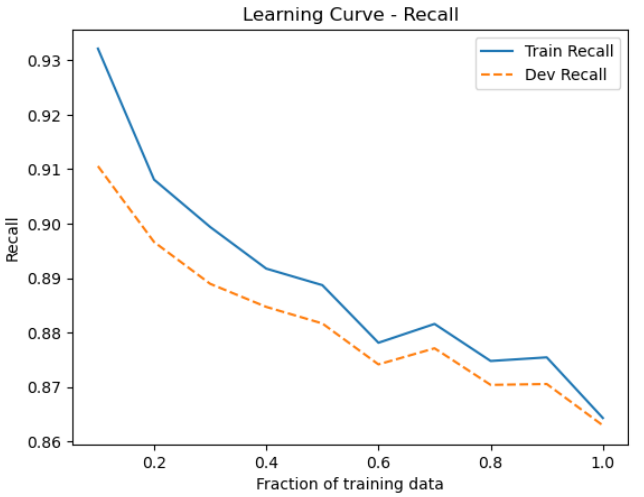


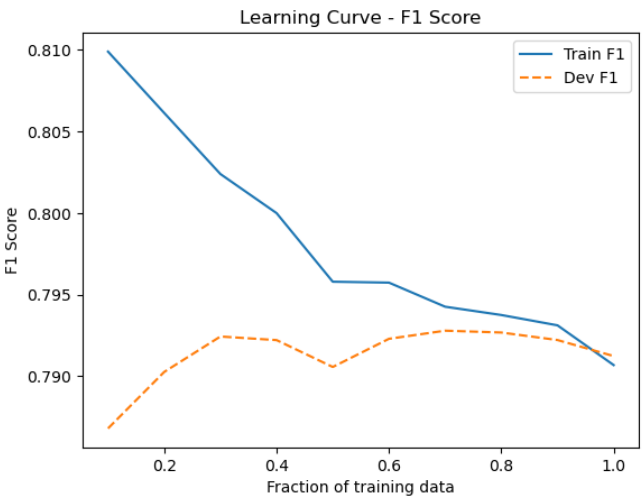




Scikit-learn Random Forest







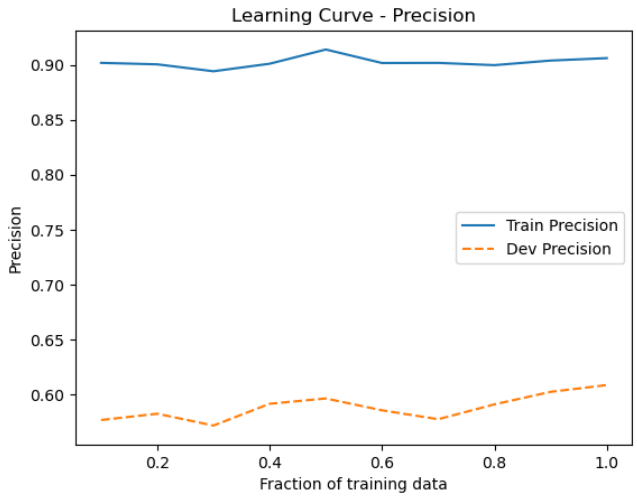
Random Forest against Scikit-learn Logistic Regression

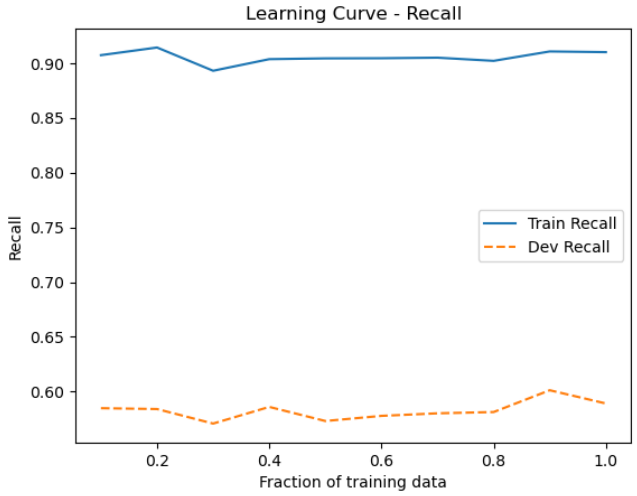
```
Logistic Regression - Test set evaluation:
              precision    recall  f1-score   support

           0       0.84      0.82      0.83     12500
           1       0.82      0.84      0.83     12500

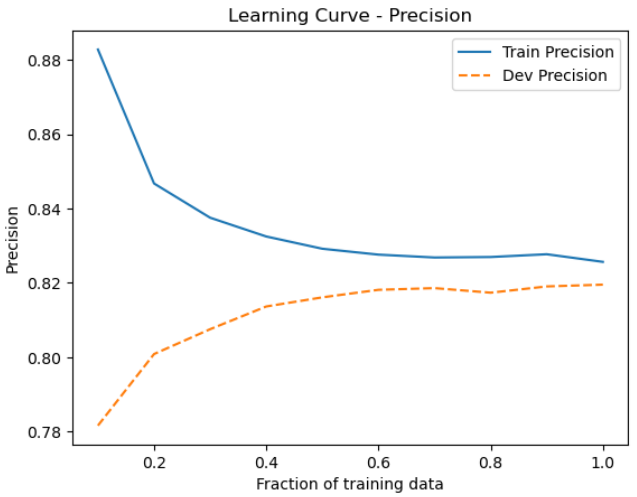
    accuracy                           0.83     25000
   macro avg       0.83      0.83      0.83     25000
weighted avg       0.83      0.83      0.83     25000
```

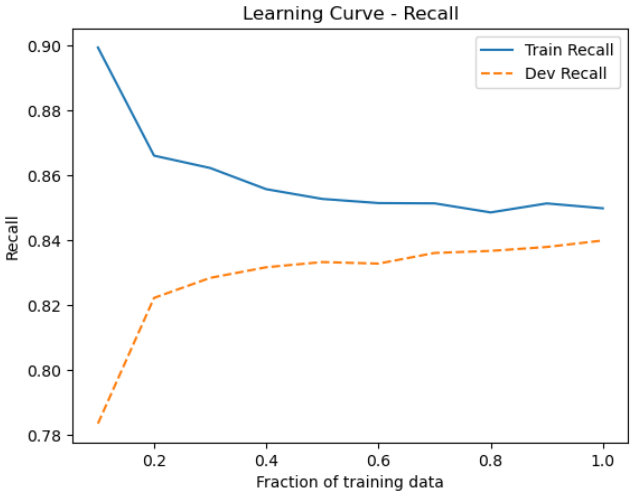
Random Forest

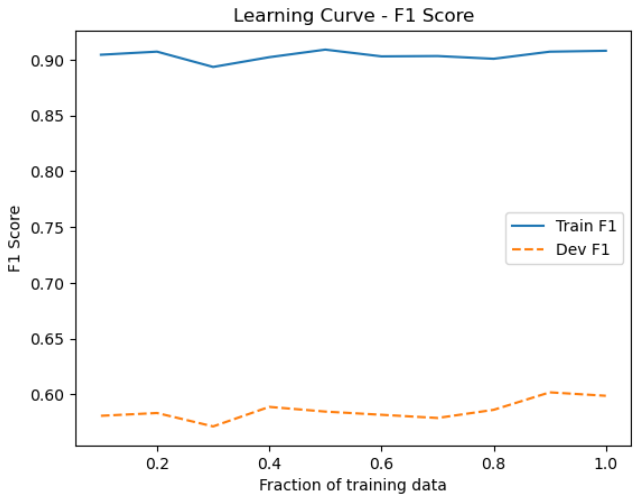


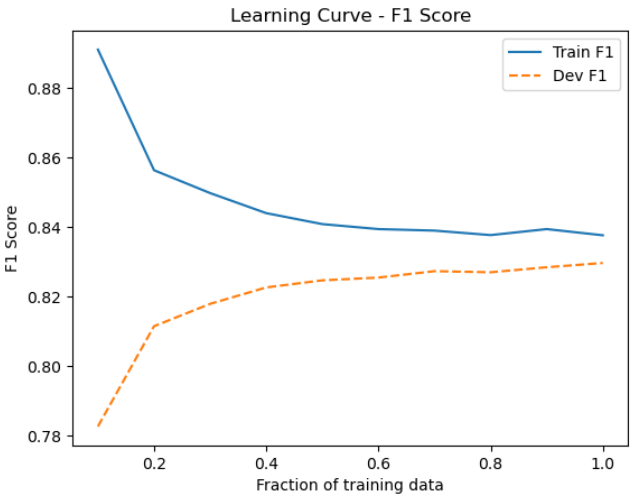


Logistic Regression









Logistic Regression

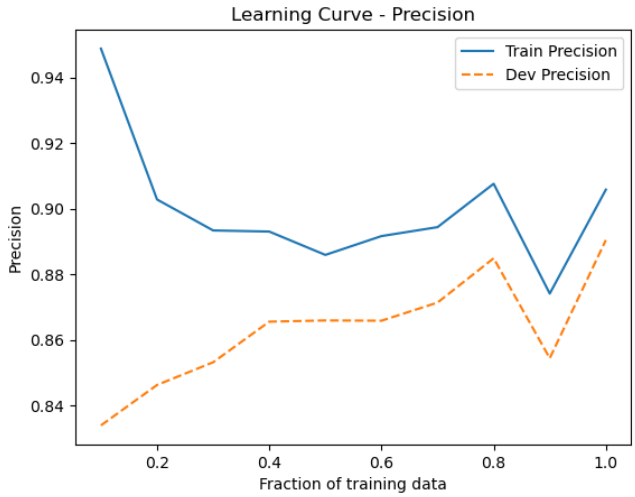
Παρακάτω βλέπουμε την επίδοση της υλοποίησης του αλγορίθμου Random Forest, όπως τον υλοποιήσαμε εμείς, σε σχέση με τις επιδόσεις των Bernoulli Naive Bayes, Random Forest και Logistic Regression του Scikit-learn:

Logistic Regression against Scikit-learn Bernoulli Naive Bayes

Scikit-learn Bernoulli Naive Bayes - Test set evaluation:

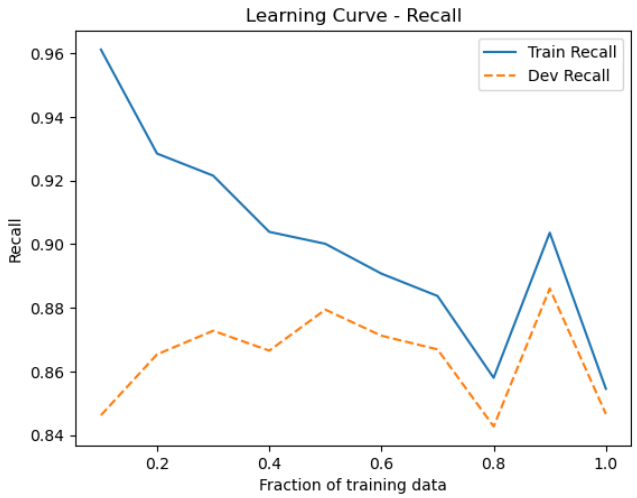
	precision	recall	f1-score	support
0	0.86	0.83	0.85	12500
1	0.84	0.87	0.85	12500
accuracy			0.85	25000
macro avg	0.85	0.85	0.85	25000
weighted avg	0.85	0.85	0.85	25000

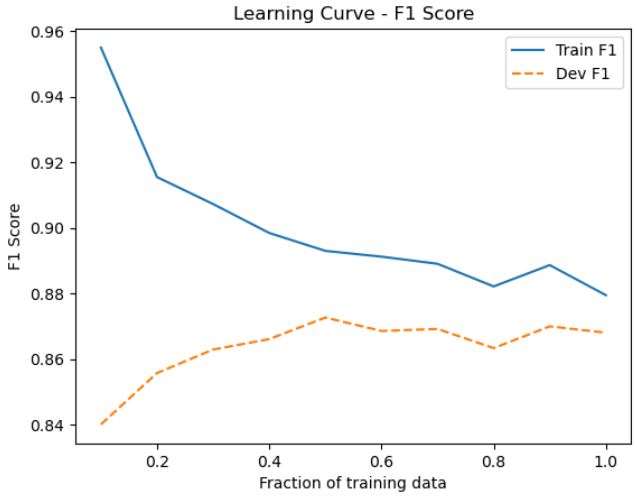
Logistic Regression

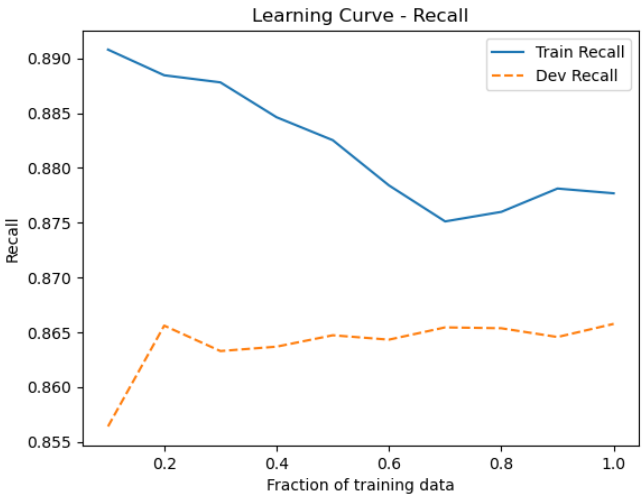


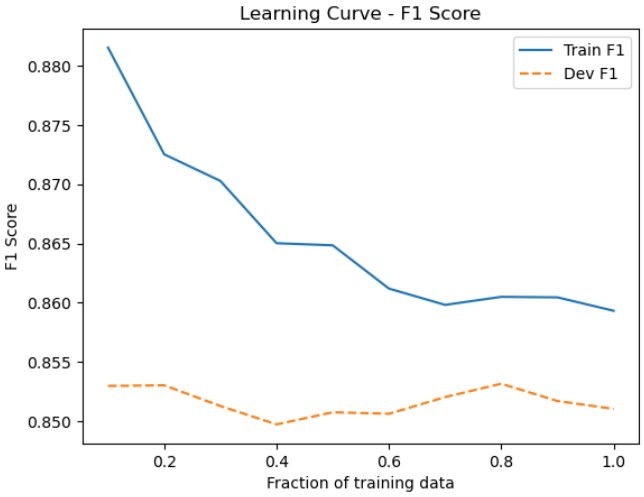
Scikit-learn Bernoulli Naive Bayes









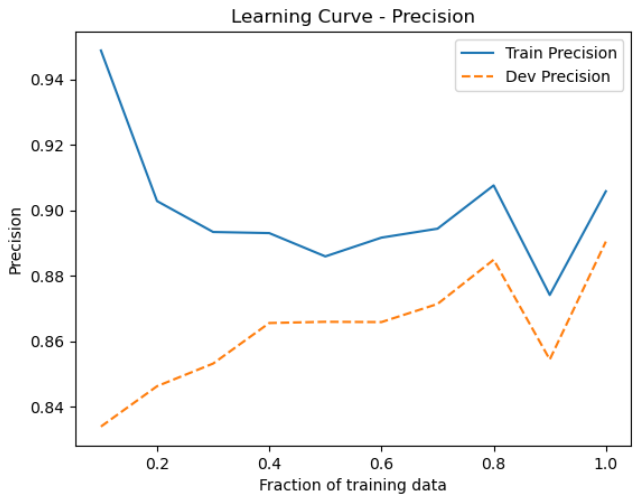


Logistic Regression against Scikit-learn Random Forest

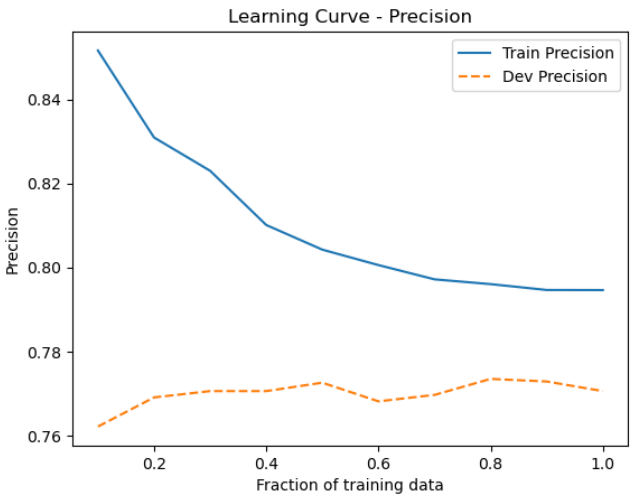
Random Forest Classifier - Test set evaluation:

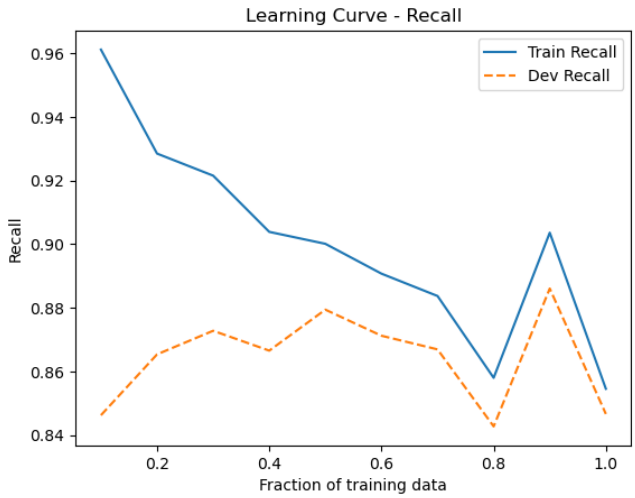
	precision	recall	f1-score	support
0	0.84	0.74	0.79	12500
1	0.77	0.86	0.81	12500
accuracy			0.80	25000
macro avg	0.81	0.80	0.80	25000
weighted avg	0.81	0.80	0.80	25000

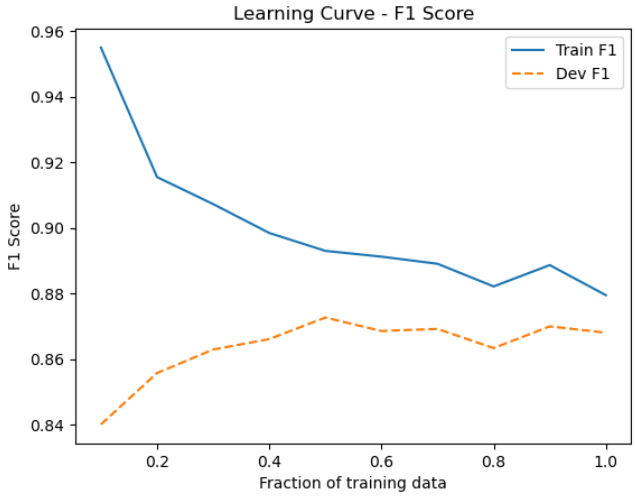
Logistic Regression

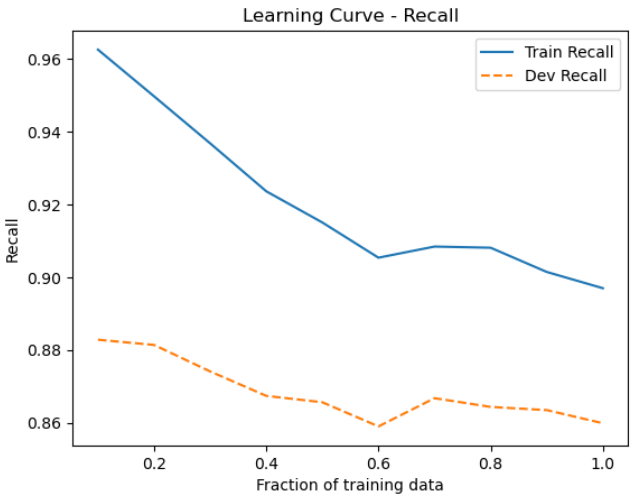


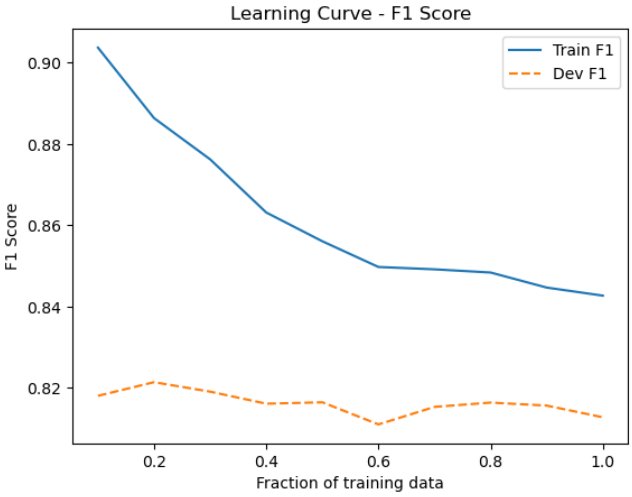
Scikit-learn Random Forest









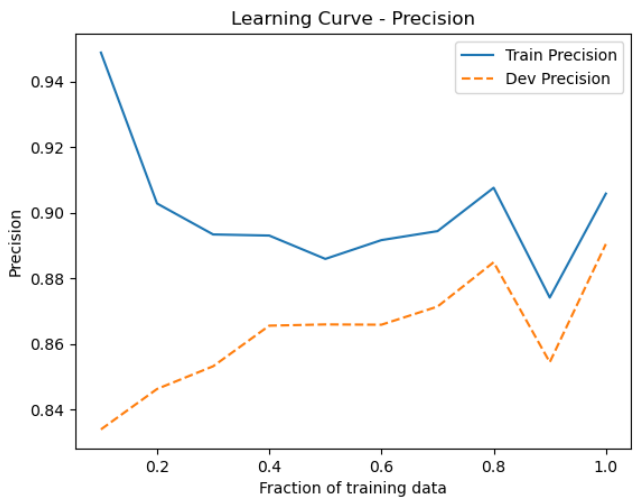


Logistic Regression against Scikit-learn Logistic Regression

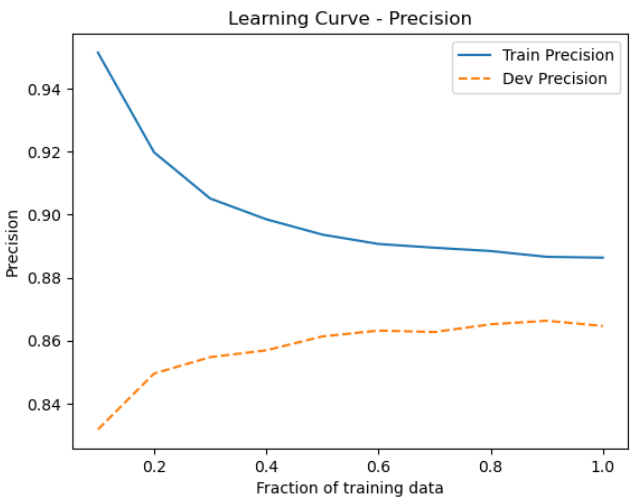
Logistic Regression - Test set evaluation:

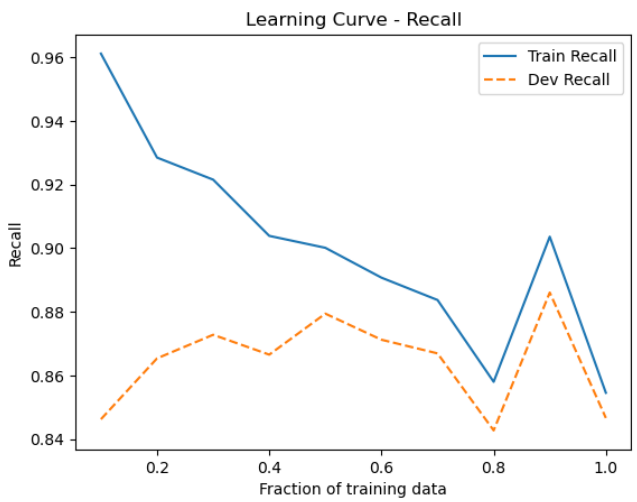
	precision	recall	f1-score	support
0	0.88	0.86	0.87	12500
1	0.86	0.88	0.87	12500
accuracy			0.87	25000
macro avg	0.87	0.87	0.87	25000
weighted avg	0.87	0.87	0.87	25000

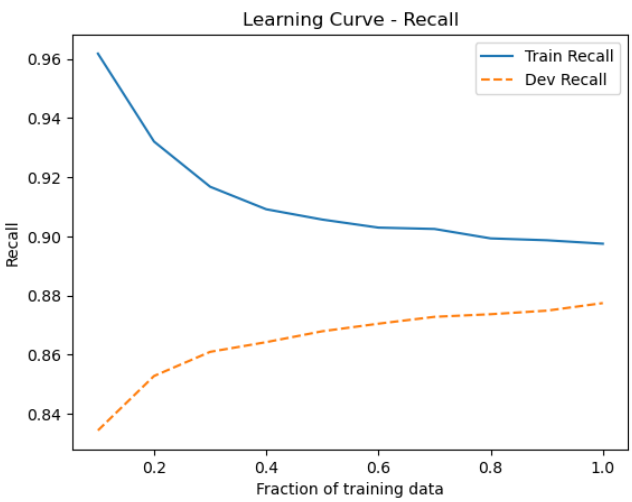
Logistic Regression

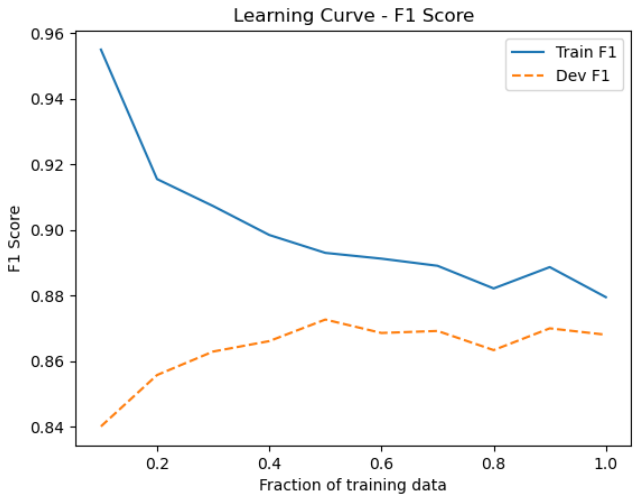


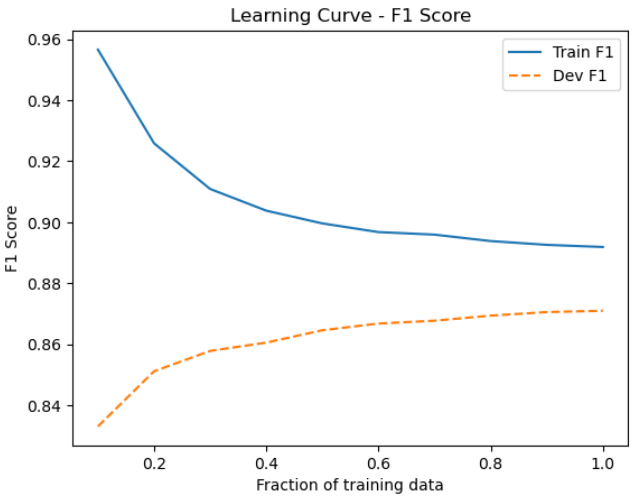
Scikit-learn Logistic Regression











Μέρος Γ'

Για το μέρος αυτό υλοποιήσαμε ένα στοιβαγμένο διπλής κατεύθυνσης RNN (stacked bidirectional RNN) υλοποιημένο σε PyTorch.8. Κάναμε χρήση του Adam optimizer και χρησιμοιποιήσαμε έτοιμες ενθέσεις λέξεων (word embeddings). Βασιστήκαμε στον αντίστοιχο κώδικα του εργαστηρίου, αντικαθιστώντας το fetch_20newsgroups το IMDB dataset που το φορτώσαμε από το keras.

Βήματα

Εισαγωγή βιβλιοθηκών: χρησιμοποιούνται βιβλιοθήκες όπως torch, torch.nn, torch.optim για τη δημιουργία και εκπαίδευση του μοντέλου, gensim για τη φόρτωση προεκπαιδευμένων Word2Vec embeddings, tensorflow.keras.preprocessing.sequence για τη διαχείριση των δεδομένων, βιβλιοθήκες αξιολόγησης από το sklearn (π.χ., accuracy_score, f1_score).

Έλεγχος διαθεσιμότητας GPU: Ορίζεται η συσκευή (device) στην οποία θα τρέξει ο κώδικας. Αν υπάρχει GPU, θα χρησιμοποιηθεί, διαφορετικά θα εκτελεστεί στη CPU.

Φόρτωση και προεπεξεργασία του IMDB dataset: Το IMDB dataset φορτώνεται από το keras.datasets.imdb. Περιέχει κριτικές ταινιών με ετικέτες positive και negative, α δεδομένα είναι ήδη σε μορφή ακολουθιών λέξεων που έχουν μετατραπεί σε αριθμητικούς δείκτες λέξεων, οι ακολουθίες περνούν από την pad_sequences ώστε να έχουν σταθερό μήκος (300 λέξεις).

Φόρτωση Word2Vec embeddings: Φορτώνεται το μοντέλο word2vec-google-news-300, δημιουργείται ένας πίνακας embeddings (embedding_matrix) που περιλαμβάνει τις αναπαραστάσεις των λέξεων, αν μια λέξη του dataset δεν υπάρχει στο Word2Vec, της αποδίδεται η μέση αναπαράσταση. Το Word2Vec είναι μια μέθοδος μετατροπής λέξεων σε διανύσματα αριθμών (embeddings) που διατηρούν τη σημασιολογική πληροφορία. Το

μοντέλο έχει εκπαιδευτεί πάνω σε τεράστιο όγκο δεδομένων (π.χ., Google News corpus) και αποδίδει διανύσματα 300 διαστάσεων για κάθε λέξη

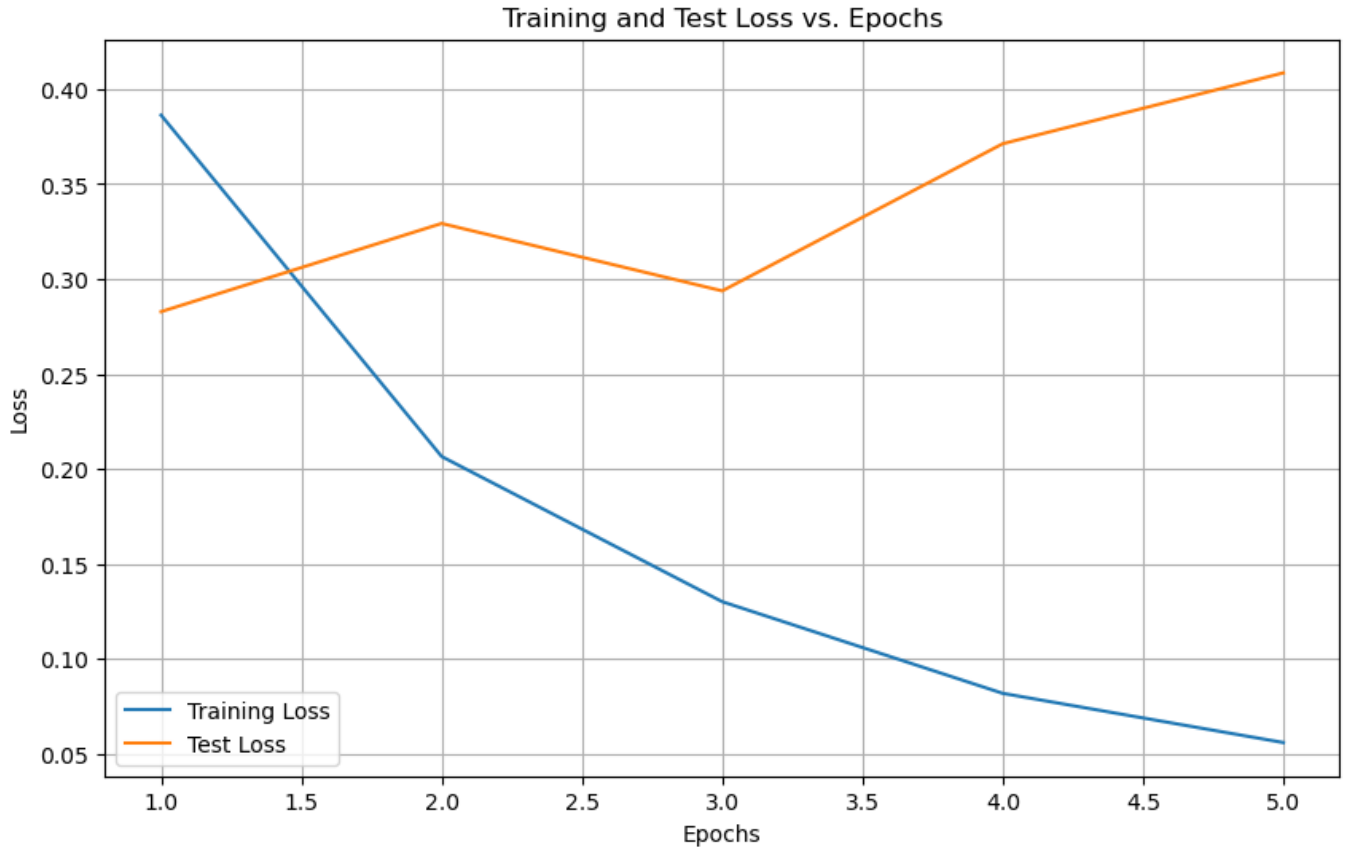
Κατασκευή Dataset και DataLoader: Ορίζεται η κλάση TextDataset για την αποθήκευση και παροχή των δεδομένων. Τα δεδομένα φορτώνονται σε DataLoader για εύκολη διαχείριση στη φάση εκπαίδευσης και αξιολόγησης.

Ορισμός του μοντέλου: Το RNN μοντέλο υποστηρίζει τρεις τύπους επαναλαμβανόμενων δικτύων: RNN, GRU, και LSTM. Περιέχει Embedding layer που φορτώνει τα προεκπαιδευμένα embeddings, Recurrent layer (LSTM στη συγκεκριμένη περίπτωση), Fully connected layer για την τελική πρόβλεψη.

Εκπαίδευση του μοντέλου: Η συνάρτηση train_model εκπαιδεύει το μοντέλο σε πολλαπλές εποχές. Υπολογίζει τη μέση απώλεια (loss) και εμφανίζει αποτελέσματα ανά εποχή.

Αξιολόγηση του μοντέλου: Η συνάρτηση evaluate_model υπολογίζει μετρικές αξιολόγησης όπως accuracy, precision, recall και f1-score.

Στην συνέχεια ακολουθεί η υλοποίηση των καμπυλών που δείχνουν το σφάλμα (loss) στα παραδείγματα εκπαίδευσης και ανάπτυξης, συναρτήσει του αριθμού των εποχών και πίνακας με αποτελέσματα ακρίβειας (precision), ανάκλησης (recall), F1 για κάθε μία από τις δύο κατηγορίες και μέσους όρους (micro- και macro-averaged), στα δεδομένα αξιολόγησης (test data):

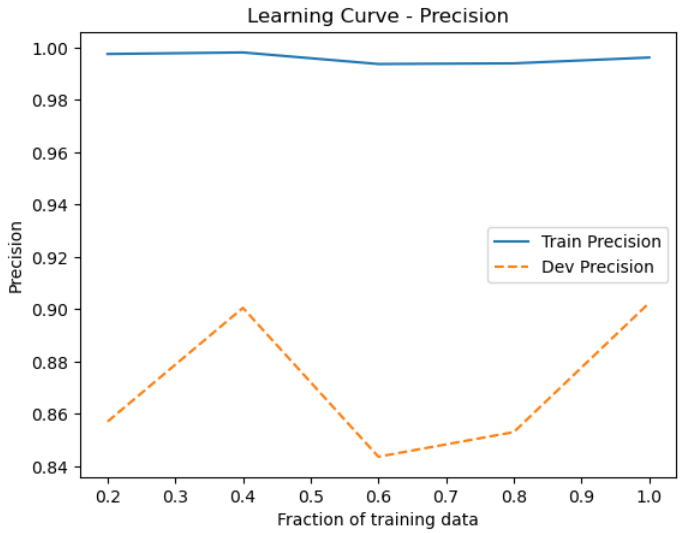


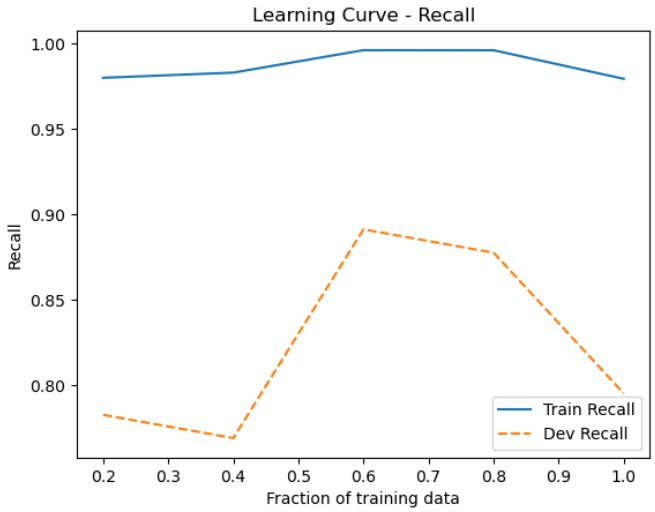
Παρατηρούμε ότι το μοντέλο αρχικά μαθαίνει καλά και μειώνει τόσο το training όσο και το test loss. Ωστόσο, μετά από ένα σημείο (γύρω στο 2^{ο}-3^{ο} epoch), παρατηρείται overfitting, με το test loss να αυξάνεται.

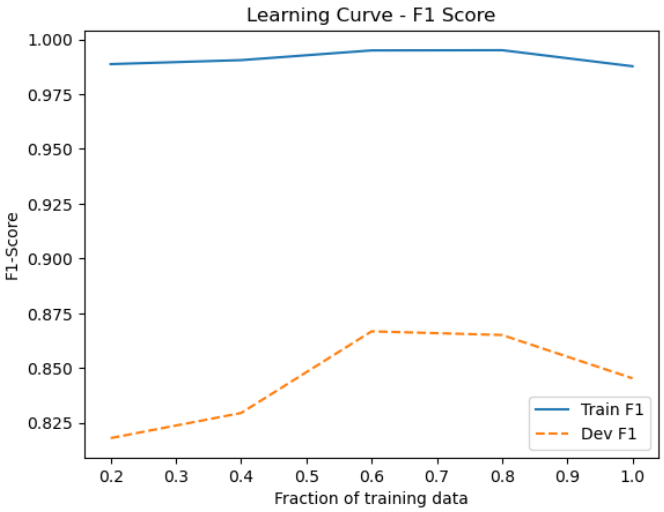
	precision	recall	f1-score	support
Negative	0.843099	0.90360	0.872302	12500.00000
Positive	0.896148	0.83184	0.862797	12500.00000
accuracy	0.867720	0.86772	0.867720	0.86772
macro avg	0.869623	0.86772	0.867549	25000.00000
weighted avg	0.869623	0.86772	0.867549	25000.00000

Παρατηρούμε ότι το μοντέλο έχει καλή συνολική απόδοση με ισορροπημένα precision και recall. Ωστόσο, παρατηρείται μικρή διαφορά στην απόδοση μεταξύ των δύο κατηγοριών, με το μοντέλο να έχει καλύτερη ευαισθησία (recall) στην κατηγορία Negative.

Τέλος σχεδιάζονται καμπύλες μάθησης που δείχνουν αποτελέσματα ακρίβειας (precision), ανάκλησης (recall), F1, για την κατηγορία των θετικών κριτικών στα δεδομένα εκπαίδευσης (training data) και ανάπτυξης (development data, συναρτήσει του πλήθους των παραδειγμάτων εκπαίδευσης που χρησιμοποιούνται σε κάθε επανάληψη του πειράματος:







Παρατηρώντας τις παραπάνω καμπύλες βλέπουμε ότι:

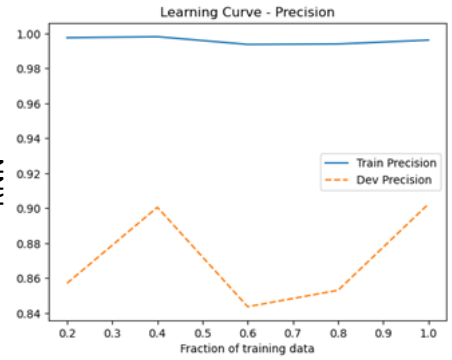
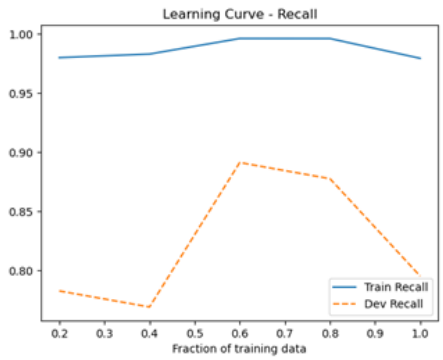
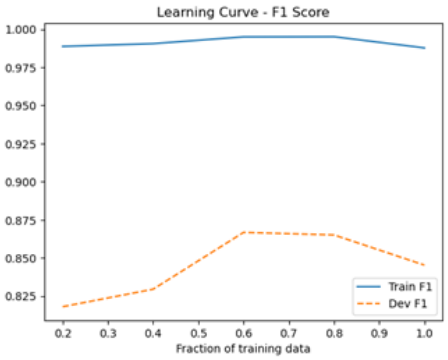
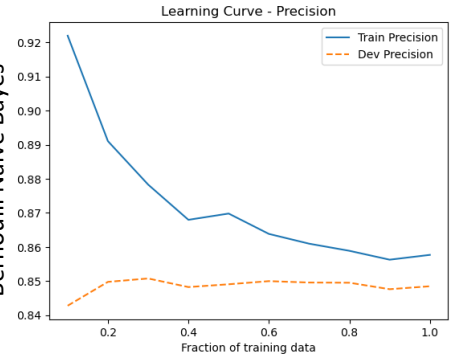
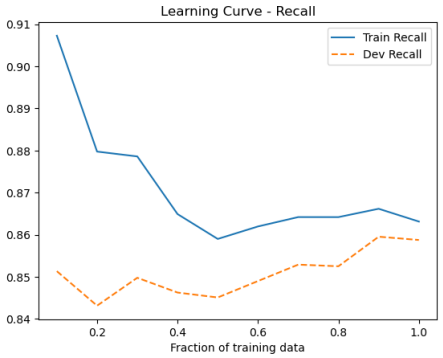
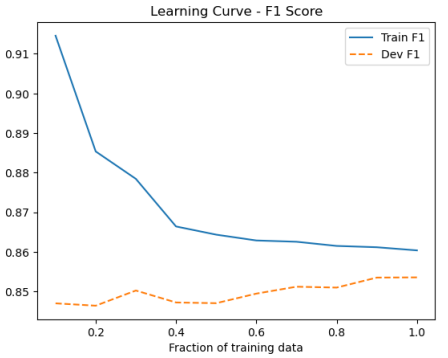
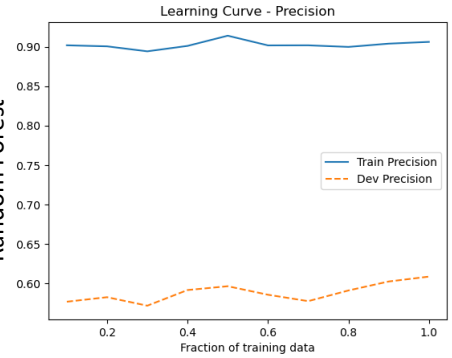
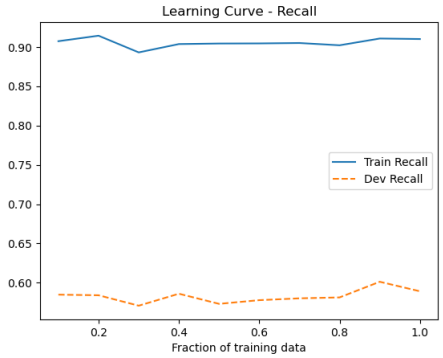
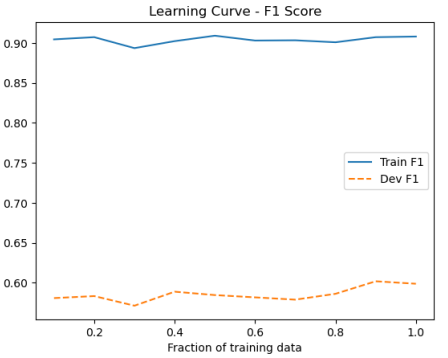
Precision: Η train precision είναι σχεδόν 1.0 σε όλες τις φάσεις εκπαίδευσης, πράγμα που δείχνει ότι το μοντέλο μαθαίνει άριστα τα δεδομένα εκπαίδευσης, πιθανώς λόγω υπερπροσαρμογής (overfitting). Η dev precision έχει αρκετές διακυμάνσεις, ιδιαίτερα στα μικρότερα ποσοστά δεδομένων. Αυτό μπορεί να οφείλεται στο γεγονός ότι το μοντέλο δεν έχει αρκετά παραδείγματα για σταθερή απόδοση και οι προβλέψεις είναι ασταθείς. Καθώς αυξάνονται τα δεδομένα εκπαίδευσης, η dev precision σταθεροποιείται και βελτιώνεται σταδιακά.

Recall: Η train recall είναι επίσης αρκετά υψηλή (κοντά στο 1.0), αλλά μειώνεται ελαφρώς καθώς αυξάνονται τα δεδομένα εκπαίδευσης, κάτι που υποδηλώνει ότι το μοντέλο χάνει μερικά παραδείγματα καθώς εκπαιδεύεται σε μεγαλύτερο dataset. Η dev recall παρουσιάζει μια βελτίωση καθώς αυξάνεται το training set. Ωστόσο, μετά από ένα σημείο (μετά το 60-70% των δεδομένων), η απόδοση αρχίζει να μειώνεται, γεγονός που δείχνει πιθανή υπερπροσαρμογή.

F1-Score: Η train F1-score παραμένει σχεδόν στο 1.0, πράγμα που υποδηλώνει πολύ κ;λή ισορροπία μεταξύ precision και recall στα δεδομένα εκπαίδευσης. Η dev F1-score ακολουθεί παρόμοια πορεία με το recall. Βελτιώνεται με την αύξηση των δεδομένων εκπαίδευσης, φτάνει σε ένα μέγιστο σημείο και στη συνέχεια μειώνεται στα μεγαλύτερα ποσοστά εκπαίδευσης.

Συγκρίσεις με τους Αλγορίθμους του Α' Μέρους

Έπειτα από δοκιμές, καταλήξαμε να επιλέξουμε για τις τιμές epochs=5, batch_size=32, layer=1

	Precision	Recall	F1 Score
RNN			
Bernoulli Naive Bayes			
Random Forest			
Logistic Regression	